

Git常用命令

01.git初始化仓库

```
1 [root@gitlab ~]#mkdir git_data
2 [root@gitlab ~]#cd git_data/
3 [root@gitlab git_data]#ll
4 total 0
5 [root@gitlab git_data]#git init
6 Initialized empty Git repository in /root/git_data/.git/
7 [root@gitlab git_data]#ll
8 total 0
9 [root@gitlab git_data]#ll -a
10 total 0
11 drwxr-xr-x  3 root root  18 Jun 11 10:57 .
12 dr-xr-x---  4 root root 174 Jun 11 10:57 ..
13 drwxr-xr-x  7 root root 119 Jun 11 10:57 .git
14
```

02.Git命令

```
1 1.查看仓库的状态(时刻保证仓库的干净)
2 [root@gitlab git_data]#git status
3 # On branch master
4 #
5 # Initial commit
6 #
7 nothing to commit (create/copy files and use "git add" to track)
8
9 2.创建普通文件
10 [root@gitlab git_data]#touch a.txt
11
12 3.将工作区的内容提交到暂存区
13 [root@gitlab git_data]#git add a.txt
14 #git add .表示所有工作区的内容提交到暂存区
15 [root@gitlab git_data]#git add .
16
17 4.删除暂存区的内容
18 [root@gitlab git_data]#git rm --cached a.txt
19 rm 'a.txt'
20
21 5.提交到暂存区和本地仓库
22 [root@gitlab git_data]#git add a.txt
23 [root@gitlab git_data]#git status
24 # On branch master
25 #
26 # Initial commit
27 #
28 # Changes to be committed:
29 #   (use "git rm --cached <file>..." to unstage)
30 #
31 #   new file:   a.txt
32 #
33
34 #使用git commit -m描述 提交到本地仓库
35 [root@gitlab git_data]#git commit -m "create a.txt"
36 [master (root-commit) b65d348] create a.txt
37 1 file changed, 0 insertions(+), 0 deletions(-)
38 create mode 100644 a.txt
39
40 查看状态:
41 [root@gitlab git_data]#git status
```

```
42 # On branch master
43 nothing to commit, working directory clean
44
45
46 创建文件      touch a.txt
47 提交到暂存区  git add a.txt
48 提交到本地仓库 git commit -m "xxx"
49
50
51 2.git删除
52
53 [root@gitlab git_data]#git rm a.txt
54 rm 'a.txt'
55 [root@gitlab git_data]#git status
56 # On branch master
57 # Changes to be committed:
58 #   (use "git reset HEAD <file>..." to unstage)
59 #
60 #   deleted:    a.txt
61 #
62 [root@gitlab git_data]#git commit -m "del a.txt"
63 [master 41fc71b] del a.txt
64 1 file changed, 0 insertions(+), 0 deletions(-)
65 delete mode 100644 a.txt
66 [root@gitlab git_data]#git status
67 # On branch master
68 nothing to commit, working directory clean
69
70
71 注意：如果工作区的文件提交到了暂存区域反悔了,删除暂存区的内容
72 [root@gitlab git_data]#touch c.txt
73 [root@gitlab git_data]#git add c.txt
74 [root@gitlab git_data]#git rm --cached c.txt
75 rm 'c.txt'
76
77
78 3.恢复工作区内容
79 [root@gitlab git_data]#git reset HEAD b.txt
80 [root@gitlab git_data]#git checkout -- b.txt
81
82
83
84 4.git比对
85 git diff  比对的是工作区域和暂存区域的不同
86 [root@gitlab git_data]#git diff
87 diff --git a/b.txt b/b.txt
88 index e69de29..ccc3e7b 100644
89 --- a/b.txt
90 +++ b/b.txt
91 @@ -0,0 +1 @@
92 +aaaaa
93
94 git diff --cached  比对的是暂存区和本地仓库的不同
95 [root@gitlab git_data]#git diff --cached
96 diff --git a/b.txt b/b.txt
97 index e69de29..ccc3e7b 100644
98 --- a/b.txt
99 +++ b/b.txt
100 @@ -0,0 +1 @@
101 +aaaaa
102
103 5.改名
104 [root@gitlab git_data]#git mv b.txt b.bak
105 [root@gitlab git_data]#git status
106 # On branch master
107 # Changes to be committed:
108 #   (use "git reset HEAD <file>..." to unstage)
109 #
110 #   renamed:    b.txt -> b.bak
111 #
```

```
112 [root@gitlab git_data]#git commit -m "newname b.txt-->b.bak"
113 [master 55335a7] newname b.txt-->b.bak
114 1 file changed, 0 insertions(+), 0 deletions(-)
115 rename b.txt => b.bak (100%)
116 [root@gitlab git_data]#git status
117 # On branch master
118 nothing to commit, working directory clean
119
```

6. 指定回滚到哪个快照

显示所有的快照：

```
122 [root@gitlab git_data]#git log --oneline
123 ae6585c xx
124 55335a7 newname b.txt-->b.bak
125 09ffe53 modifile b.txt
126 043a297 newfile b.txt
127 41fc71b del a.txt
128 b65d348 create a.txt
129 [root@gitlab git_data]#ll
130 total 8
131 -rw-r--r-- 1 root root 6 Jun 11 12:10 b.bak
132 -rw-r--r-- 1 root root 6 Jun 11 12:07 b.txt
133
```

使用git reset --hard 回滚

```
135 [root@gitlab git_data]#git reset --hard b65d348
136 HEAD is now at b65d348 create a.txt
137 [root@gitlab git_data]#ll
138 total 0
139 -rw-r--r-- 1 root root 0 Jun 11 12:12 a.txt
140
```

7. 查看历史所有的提交动作：

```
142 [root@gitlab git_data]#git reflog
143 b65d348 HEAD@{0}: reset: moving to b65d348
144 ae6585c HEAD@{1}: commit: xx
145 55335a7 HEAD@{2}: commit: newname b.txt-->b.bak
146 09ffe53 HEAD@{3}: commit: modifile b.txt
147 043a297 HEAD@{4}: commit: newfile b.txt
148 41fc71b HEAD@{5}: commit: del a.txt
149 b65d348 HEAD@{6}: commit (initial): create a.txt
150 [root@gitlab git_data]#git reset --hard ae6585c
151 HEAD is now at ae6585c xx
152 [root@gitlab git_data]#ll
153 total 8
154 -rw-r--r-- 1 root root 6 Jun 11 12:16 b.bak
155 -rw-r--r-- 1 root root 6 Jun 11 12:16 b.txt
156
157
```

Git分支

1. 查看git分支 *表示当前所在的分支

```
160 [root@gitlab git_data]#git branch
161 * master
```

2. 创建分支

```
163 [root@gitlab git_data]#git branch dev
164 [root@gitlab git_data]#git branch
165 dev
166 * master
167
```

3. 切换到dev分支

```
169 [root@gitlab git_data]#git checkout dev
170 Switched to branch 'dev'
171 [root@gitlab git_data]#git branch
172 * dev
173 master
174
```

4. 分支创建文件并且提交到本地仓库

```
176 [root@gitlab git_data]#touch aa.txt
177 [root@gitlab git_data]#git add .
178 [root@gitlab git_data]#git commit -m "newfile aa.txt"
179 [dev 2694c7f] newfile aa.txt
180 1 file changed, 0 insertions(+), 0 deletions(-)
181 create mode 100644 aa.txt
```

```
182
183 5. 合并分支
184 先切换回master分支
185 [root@gitlab git_data]#git checkout master
186 Switched to branch 'master'
187 [root@gitlab git_data]#git branch
188     dev
189 * master
190 合并dev分支到master分支
191 [root@gitlab git_data]#git merge dev
192 Updating ae6585c..2694c7f
193 Fast-forward
194   aa.txt | 0
195   1 file changed, 0 insertions(+), 0 deletions(-)
196   create mode 100644 aa.txt
197
198 6. 分支写完功能后立刻删除(因为代码会落后于远程分支)
199 [root@gitlab git_data]#git branch -d dev
200 Deleted branch dev (was 2694c7f).
201 [root@gitlab git_data]#git branch
202 * master
203
204
205 7. 冲突合并
206 -----
207 修改文件的内容不影响分支的合并
208 修改主干分支的文件增加bbbbbb
209 [root@gitlab git_data]#vim b.txt
210 aaaaa
211 bbbbbb
212
213 使用一条命令可以直接创建并切换到dev分支
214 [root@gitlab git_data]#git checkout -b dev
215 Switched to a new branch 'dev'
216 [root@gitlab git_data]#
217 [root@gitlab git_data]#
218 [root@gitlab git_data]#git branch
219 * dev
220     master
221
222 修改b.txt的内容
223 [root@gitlab git_data]#vim b.txt
224 aaaaa
225 bbccc
226
227 提交到本地仓库
228 [root@gitlab git_data]#git add .
229 [root@gitlab git_data]#git commit -m "modifile bbccc"
230
231 返回到master分支将dev分支合并
232 [root@gitlab git_data]#git checkout master
233 Switched to branch 'master'
234 [root@gitlab git_data]#git branch
235     dev
236 * master
237 [root@gitlab git_data]#cat b.txt
238 aaaaa
239 bbbbbb
240 [root@gitlab git_data]#git merge dev
241 Updating f56faba..dc17f6d
242 Fast-forward
243   b.txt | 2 +
244   1 file changed, 1 insertion(+), 1 deletion(-)
245 [root@gitlab git_data]#cat b.txt
246 aaaaa
247 bbccc
248 -----
249
250 影响冲突合并的案例
251 创建dev分支
```

```
252 [root@gitlab git_data]#git branch dev
253
254 在master分支修改b.txt第3行
255 [root@gitlab git_data]#cat b.txt
256 aaaaa
257 iiii
258 bbbbb
259 [root@gitlab git_data]#git commit -am "modifile b.txt bbbbb"
260 [master ae4ab87] modifile b.txt bbbbb
261 1 file changed, 1 insertion(+)
262
263 在切换到dev分支修改b.txt第3行
264 [root@gitlab git_data]#cat b.txt
265 aaaaa
266 iiii
267 ccccc
268
269 提交到本地仓库
270 [root@gitlab git_data]#git commit -am "modifile b.txt ccccc"
271 [dev 925c06c] modifile b.txt ccccc
272 1 file changed, 1 insertion(+)
273
274
275 在切换到master分支进行分支合并
276 [root@gitlab git_data]#git checkout master
277 Switched to branch 'master'
278
279 [root@gitlab git_data]#git branch
280 dev
281 * master
282 [root@gitlab git_data]#git merge dev
283 Auto-merging b.txt
284 CONFLICT (content): Merge conflict in b.txt
285 Automatic merge failed; fix conflicts and then commit the result.
286
287 查看文件显示代码冲突
288 [root@gitlab git_data]#cat b.txt
289 aaaaa
290 iiii
291 <<<<<< HEAD
292 bbbbb
293 =====
294 ccccc
295 >>>>>> dev
296
297
298
299 手动解决代码冲突 留下一行或者全部留下
300 [root@gitlab git_data]#vim b.txt
301 aaaaa
302 iiii
303 bbbbb
304 ccccc
305
306
307 [root@gitlab git_data]#git commit -am "merge dev"
308 [master 325eef2] merge dev
309 [root@gitlab git_data]#
310 [root@gitlab git_data]#
311 [root@gitlab git_data]#git status
312 # On branch master
313 nothing to commit, working directory clean
314 [root@gitlab git_data]#cat b.txt
315 aaaaa
316 iiii
317 bbbbb
318 ccccc
319
320 合并后删除dev分支
321 [root@gitlab git_data]#git branch -d dev
```

```
322 Deleted branch dev (was 925c06c).
323
324
325 小结:
326 git初始化
327 mkdir git_data
328 cd git_data
329 git init
330 git status
331 git add .
332 git commit -m "描述"
333 git rm --cached
334 git mv
335 git rm
336 git diff
337 git diff --cached
338 git reflog
339 git reset --hard 325eef2
340 git branch
341 git checkout dev
342 git branch -d dev
343 git merge
```

Gitlab服务

01. Gitlab简介

1. **GitLab** 是一个用于仓库管理系统的开源项目。使用**Git**作为代码管理工具，并在此基础上搭建起来的**web**服务。可通过**web**界面进行访问公开的或者私人项目。它拥有与**Github**类似的功能，能够浏览源代码，管理缺陷和注释。可以管理团队对仓库的访问，它非常易于浏览提交过的版本并提供一个文件历史库。团队成员可以利用内置的简单聊天程序(**wall**)进行交流。它还提供一个代码片段收集功能可以轻松实现代码复用。
2. 官网: <https://about.gitlab.com/>
3. 国内镜像: <https://mirrors.tuna.tsinghua.edu.cn/gitlab-ce/yum/>

02.Gitlab部署

```
1 一.环境要求
2 1、CentOS7
3 2、4G内存（实验）生产（至少8G）
4 3、安装包: gitlab-ce-16.9.8-ce
5 4、禁用防火墙，关闭selinux
6
7 二.下载gitlab的rpm包
8 [root@gitlab ~]# wget https://mirrors.tuna.tsinghua.edu.cn/gitlab-ce/yum/e17/gitlab-ce-16.9.8-
ce.0.e17.x86_64.rpm
9
10 三.安装gitlab
11 # 安装过程中提示需要安装依赖
12 [root@gitlab ~]# rpm -ivh gitlab-ce-16.9.8-ce.0.e17.x86_64.rpm
13 warning: gitlab-ce-16.9.8-ce.0.e17.x86_64.rpm: Header V4 RSA/SHA1 Signature, key ID f27eab47: NOKEY
14 error: Failed dependencies:
15     policycoreutils-python is needed by gitlab-ce-16.9.8-ce.0.e17.x86_64
16 # 安装依赖
17 [root@gitlab ~]# yum -y install policycoreutils-python
18
19 # 安装完依赖继续安装gitlab
20 [root@gitlab ~]# rpm -ivh gitlab-ce-16.9.8-ce.0.e17.x86_64.rpm
21 warning: gitlab-ce-16.9.8-ce.0.e17.x86_64.rpm: Header V4 RSA/SHA1 Signature, key ID f27eab47: NOKEY
22 Preparing... ##### [100%]
23 Updating / installing...
```

```
24 1:gitlab-ce-16.9.8-ce.0.e17 ##### [100%]
25
26
27 四.启动前配置访问域名
28 [root@gitlab ~]# vim /etc/gitlab/gitlab.rb
29 ...
30 external_url 'http://10.0.0.200'
31 ...
32 配置完成后执行命令:
33 [root@gitlab ~]# gitlab-ctl reconfigure
34
```

03. Gitlab组件

```
1 nginx:          静态web服务器
2 gitlab-workhorse: 轻量级的反向代理服务器
3 logrotate:       日志文件管理工具
4 postgresql:      数据库
5 redis:           缓存数据库
6 sidekiq:         用于在后台执行队列任务（异步执行）。(Ruby)
7 unicorn: An HTTP server for Rack applications, GitLab Rails应用是托管在这个服务器上面的。(Ruby Web Server, 主要使用Ruby编写)
```

04. Gitlab常用命令

```
1 gitlab-ctl start          # 启动gitlab 在执行gitlab-ctl reconfigure 服务默认已经启动
2 gitlab-ctl status         # 查看目前gitlab所有服务运维状态
3 gitlab-ctl stop           # 停止gitlab服务
4 gitlab-ctl stop nginx     # 单独停止某个服务
5 gitlab-ctl tail           # 查看所有服务的日志
```

05. Gitlab页面配置

登陆gitlab: 10.0.0.200

一. 登陆密码

执行gitlab-ctl reconfigure完成后会提示密码存放的位置:

```
Notes:
Default admin account has been configured with following details:
Username: root
Password: You didn't opt-in to print initial root password to STDOUT.
Password stored to /etc/gitlab/initial_root_password. This file will be cleaned up in first re
configure run after 24 hours.

NOTE: Because these credentials might be present in your log files in plain text, it is highly
recommended to reset the password following https://docs.gitlab.com/ee/security/reset\_user\_password.html#reset-your-root-password.

[2024-05-14T14:31:19+08:00] WARN: This release of Cinc Client became end of life (EOL) on May
1st 2024. Please update to a supported release to receive new features, bug fixes, and securit
y updates.
gitlab Reconfigured!
```

```
[root@gitlab ~]# cat /etc/gitlab/initial_root_password
# WARNING: This value is valid only in the following conditions
#
#   1. If provided manually (either via `GITLAB_ROOT_PASSWORD` environment variable or
#   via `gitlab_rails['initial_root_password']` setting in `gitlab.rb`, it was provided before dat
#   abase was seeded for the first time (usually, the first reconfigure run).
#
#   2. Password hasn't been changed manually, either via UI or via command line.
#
#   If the password shown here doesn't work, you must reset the admin password followin
#   g https://docs.gitlab.com/ee/security/reset_user_password.html#reset-your-root-password.
Password: gPbqDBwjrhIJd2SJytM4r30EC6K5ZLBHj/0teVFpeMU=
# NOTE: This file will be automatically deleted in the first reconfigure run after 24 hours.
```

← → ↻ ▲ 不安全 | 10.0.0.200/users/sign_in 🔍 🏠 ☆



用户名或主要电子邮件

root

密码

.....

[忘记密码?](#)

☐ 记住账号

登录

还没有账户? [立即注册](#)

二. 配置页面显示为中文

← → ↻ ▲ 不安全 | 10.0.0.200





Administrator
@root

Set status

Edit profile

Preferences

Sign out

Merge requests >

To-Do List

Milestones

Snippets

Activity

Your work / Projects

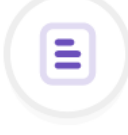
 **Check your sign-up restrictions**

Your GitLab instance allows anyone to register for an account, which is a security r
expected to register for an account.

Deactivate

Acknowledge

Well
Faster rel

 **Create a project**

Projects are where you store your code, access
issues, wiki and other features of GitLab.

点击偏好设置

← → ↺

10.0.0.200/-/profile/preferences

🔍 Search or go to...

User settings

Profile

Account

Applications

Chat

Access Tokens

Emails

Password

Notifications

SSH Keys

GPG Keys

Preferences

Comment Templates

Active Sessions

Authentication Log

Help

Admin Area

User Settings / Preferences

Behavior

Tab width

8

Must be a number between 1 and 12

Localization

Customize language and region related settings. [Learn more.](#)

Language

Chinese, Simplified - 简体中文 (98% translated)

This feature is experimental and translations are not yet complete. [Help translate GitLab into your language](#)

First day of the week

System default (Sunday)

Time preferences

Configure how dates and times display for you. [Learn more.](#)

☒ Use relative times

For example: 30 minutes ago.

Time format

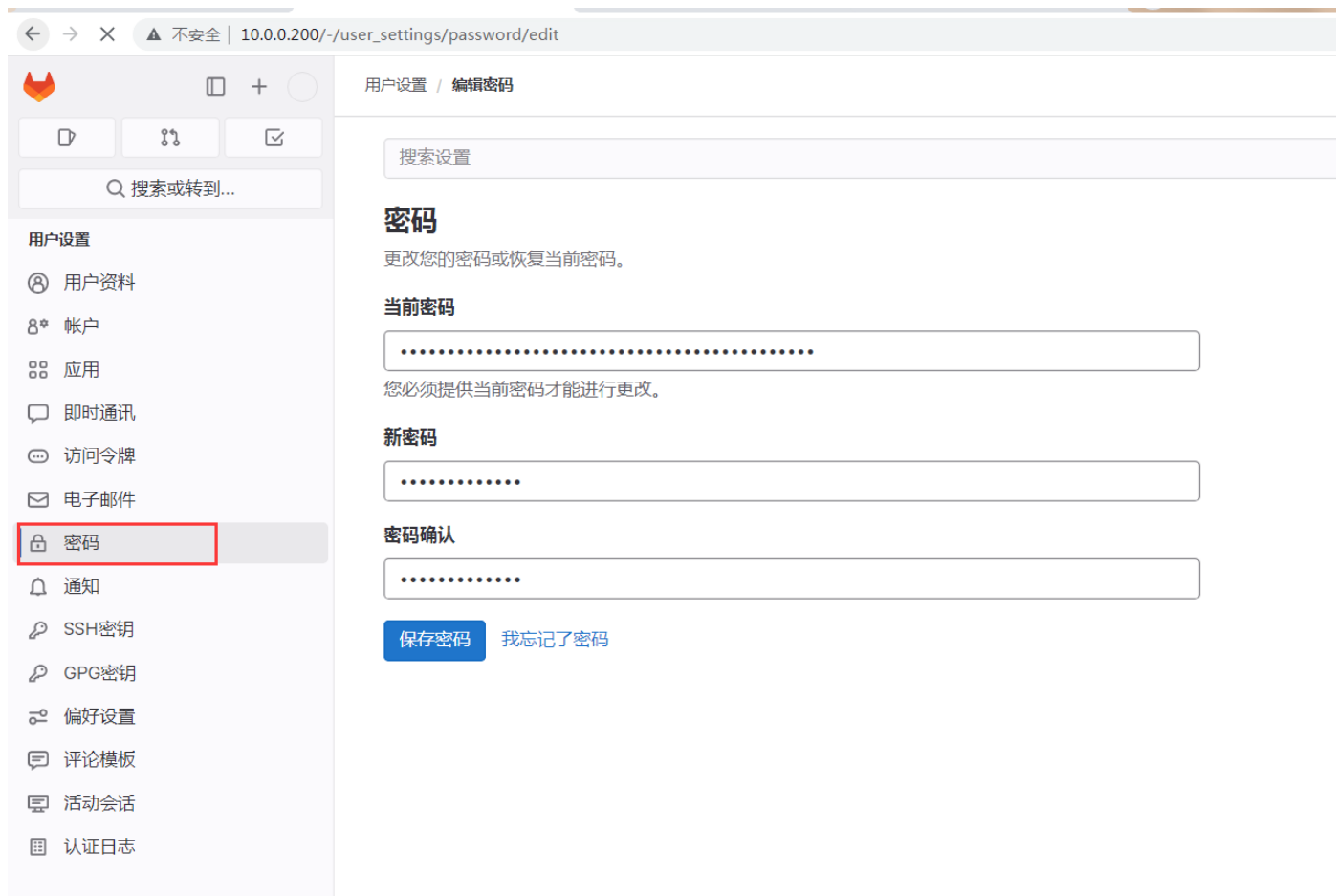
☒ System

Save changes

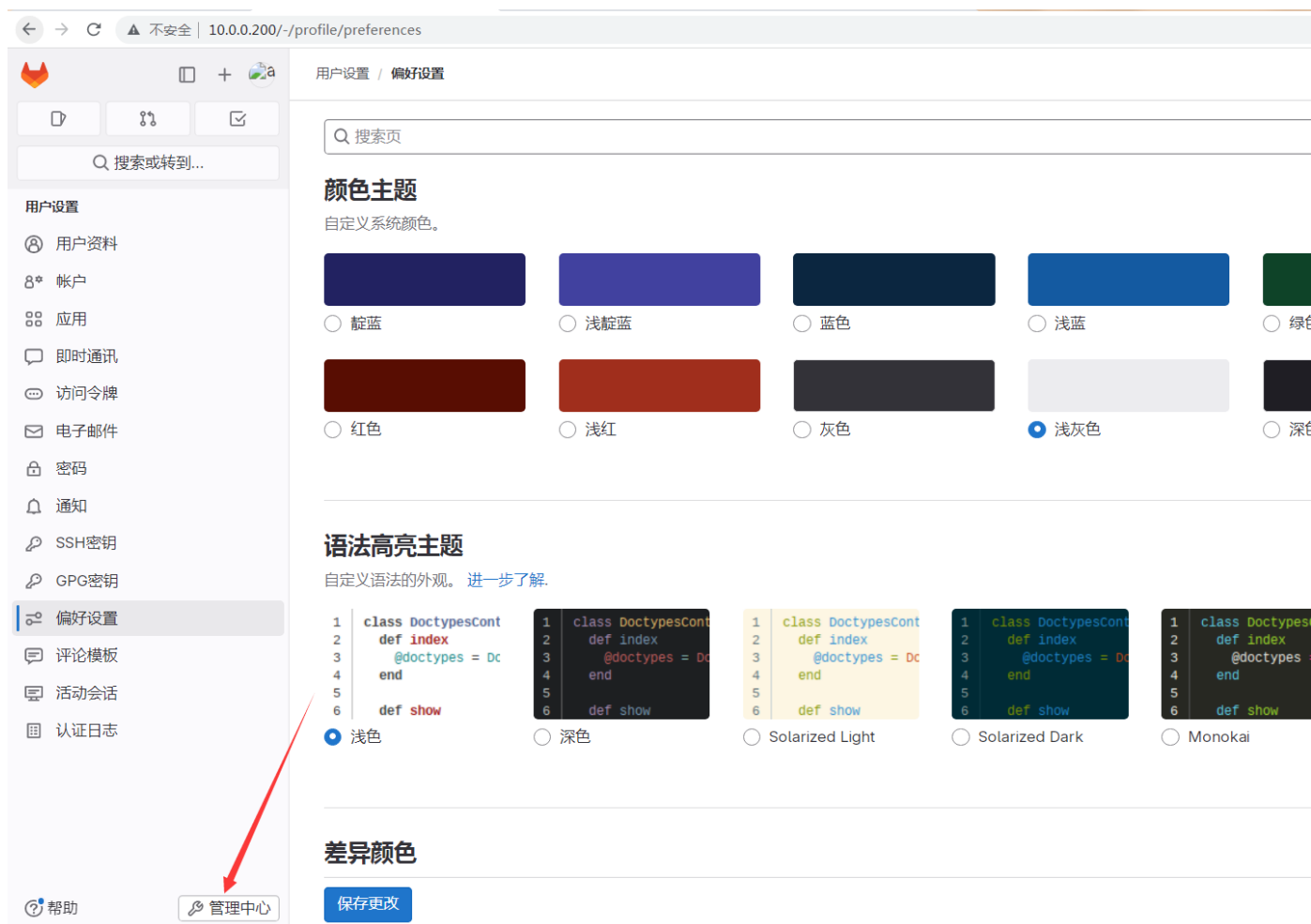
语言选项设置为简体中文

然后保存改变

三. 修改默认密码



四. 开启钩子本地网络请求



← → × 10.0.0.200/admin

🔍 搜索或转到...

管理中心

仪表盘

项目

用户

群组

主题

Gitaly服务器

CI/CD

分析

监控

消息

系统钩子

应用程序

滥用报告

部署密钥

标记

设置

管理中心 / 仪表盘

💡 从 GitLab 获取安全更新并保持更新

注册 GitLab Security Newsletter 以获得安全更新通知。

注册 GitLab 通讯

实例概览

📁 项目

0

新建项目

查看最新项目

👤 用户

1

查看最新的用户 / 月

统计

通用

集成

仓库

CI/CD

报告

指标与分析

网络

外观

偏好设置

功能

注册

LDAP

Gravatar

OmniAuth

通过电子邮件回复

容器镜像库

GitLab Pages

Instance Runners

← → 10.0.0.200/admin/application_settings/network

🔍 搜索或转到...

分析

监控

消息

系统钩子

应用程序

滥用报告

部署密钥

标记

设置

通用

集成

仓库

CI/CD

报告

指标与分析

网络

外观

偏好设置

管理中心 / 网络

Git LFS 速率限制

配置取代一般用户和 IP 率限制的 Git LFS 请求的特定限制。

展开

Git SSH 操作速率限制

限制用户每分钟每个仓库可以执行的 Git 操作数

展开

出站请求

允许来自 hooks 和集成对本地网络的请求。

☐ 阻止所有请求，但白名单中定义的 IP 地址、IP 范围和域名除外

☒ 允许来自 webhooks 和集成对本地网络的请求

☒ 允许系统钩子向本地网络发送请求

Webhook 和服务可以访问的本地 IP 地址和域名。

example.com, 192.168.1.1, xn--itlab-qla.com

即使不允许本地请求，也可以向这些 IP 地址和域名发出请求。支持 IP 范围，例如 1:0:0:0:0:0/124 和 127.0.0.0/28。不支持域名通配符。要分隔条目，请使用逗号、分号或换行符。许可名单最多可包含 1000 个条目。域名必须是 IDNA 编码的。

☒ 强制执行 DNS 重新绑定攻击保护

较新版本的Gitlab需要勾选此项,开启钩子的本地网络请求

06. Gitlab实战

一. 创建组



← → ↻

⚠ 不安全 | 10.0.0.200/groups/new#create-group-pane

🏠 + 🖼 a

📄 🔗 ☑

🔍 搜索或转到...

你的工作

📁 项目

👤 群组

📄 议题

🔗 合并请求

☑ 待办事项列表

🕒 里程碑

✂ 代码片段

🕒 动态

🔍 帮助

🔑 管理中心

您的工作 / 群组 / 新建群组 / 创建群组

ops游戏组

组名必须以字母、数字、表情或下划线开头，可以包含句点、破折号、空格和括号。

⚠ 如果您打算使用 SCIM 集成，您的组名称不得包含句点，因为这可能会导致错误。

群组 URL

http://10.0.0.200/

ops

群组路径可用。

自定义组的名称

可见性级别

哪些人可以看到这个群组？ [查看文档](#)

☒ 🔒 私有

群组及其项目只能由成员查看。

☐ 🔒 内部

除外部用户外，任何登录用户均可查看该群组和任何内部项目。

☐ 🌐 公开

群组和任何公开项目可以在没有任何身份验证的情况下查看。

现在，个性化您的 GitLab 体验

我们将使用它来帮助向您展示正确的功能和信息。

角色

软件开发人员

▼

谁将使用这个群组？

☐ 我的公司或团队 ☐ 仅我自己

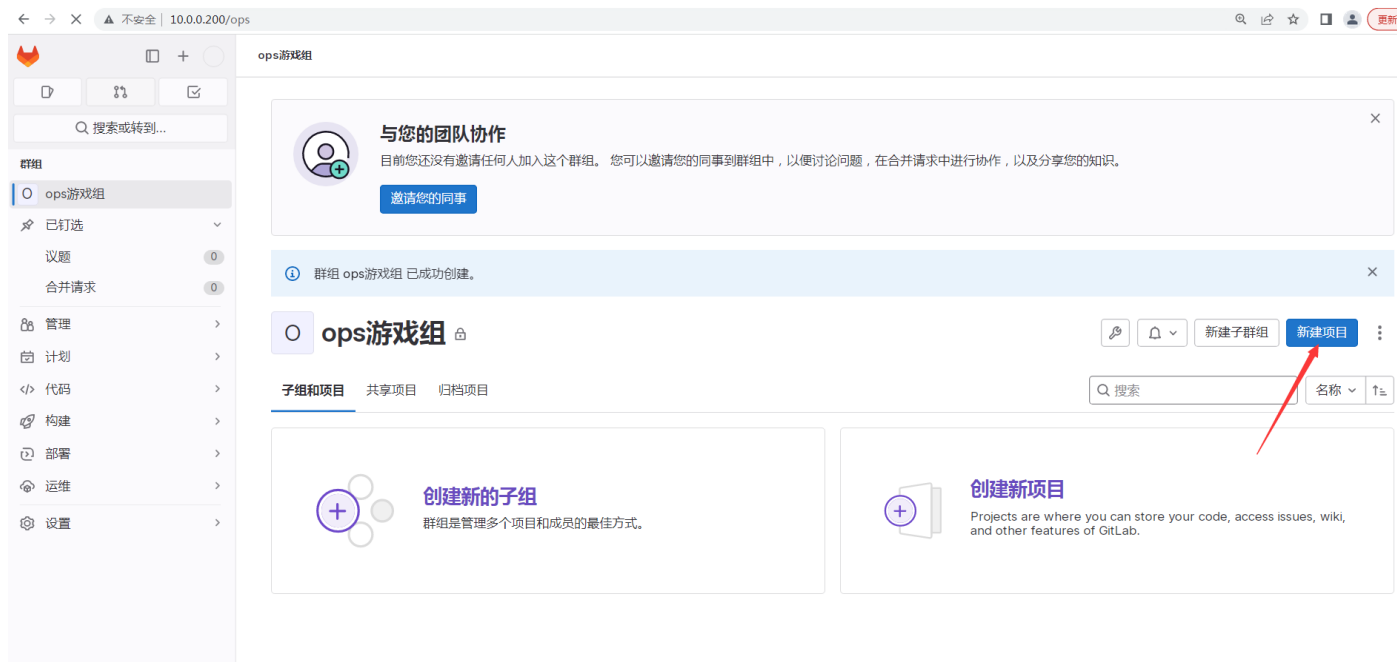
你会用这个群组做什么？

▼

创建群组

取消

二. 基于组创建项目



注意: 创建的项目属于ops小组



创建空白项目

创建一个空白项目来存放您的文件，规划您的工作，并在代码等方面进行协作。

项目名称

必须以小写或大写字母、数字、表情符号或下划线开头。也可以包含点、加号、破折号或空格。

项目 URL

项目标识串

可见性级别

☒ 私有

项目访问权限必须明确授予每个用户。如果此项目是一个群组的一部分，访问权限将授予该群组的成员。

项目配置

☐ 使用自述文件初始化仓库

允许您立即克隆这个项目的仓库。如果您计划推送一个现有的仓库，请跳过这个步骤。

☐ 启用静态应用安全测试 (SAST)

分析源代码查找已知安全漏洞。 [了解更多](#)。

至此第一个代码仓库就创建完成了,不过里面是空的。接下来上传代码

三. 将代码提交到项目中存储

项目创建完成后,需要将游戏代码上传到此项目

服务器和代码仓库连接方式有两种

第一种: 基于用户名和密码的方式连接

第二种: 基于SSH免秘钥方式(和命令行的SSH免秘钥相同)

我们使用基于免秘钥方式: 方便+安全性高

1.生成密钥对(在这这是直接使用的gitlab服务器)

```
1 [root@gitlab ~]# ssh-keygen # 输入此命令下面选项一律按回车即可
2 Generating public/private rsa key pair.
3 Enter file in which to save the key (/root/.ssh/id_rsa):
4 Created directory '/root/.ssh'.
5 Enter passphrase (empty for no passphrase):
6 Enter same passphrase again:
7 Your identification has been saved in /root/.ssh/id_rsa.
8 Your public key has been saved in /root/.ssh/id_rsa.pub.
9 The key fingerprint is:
10 SHA256:rgpwsistAdhPmaDLuMwvkhTzv0hOTjccvjw+nXHYZW0 root@gitlab
11 The key's randomart image is:
12 +---[RSA 2048]----+
13 |                 |
14 | . . +. +        |
15 | .o .*oo..       |
16 |+o. ++.O. o o .  |
17 |oo+o .+So o o E  |
18 |oo..o.o.o + .   |
19 |+o o.+..o o      |
```

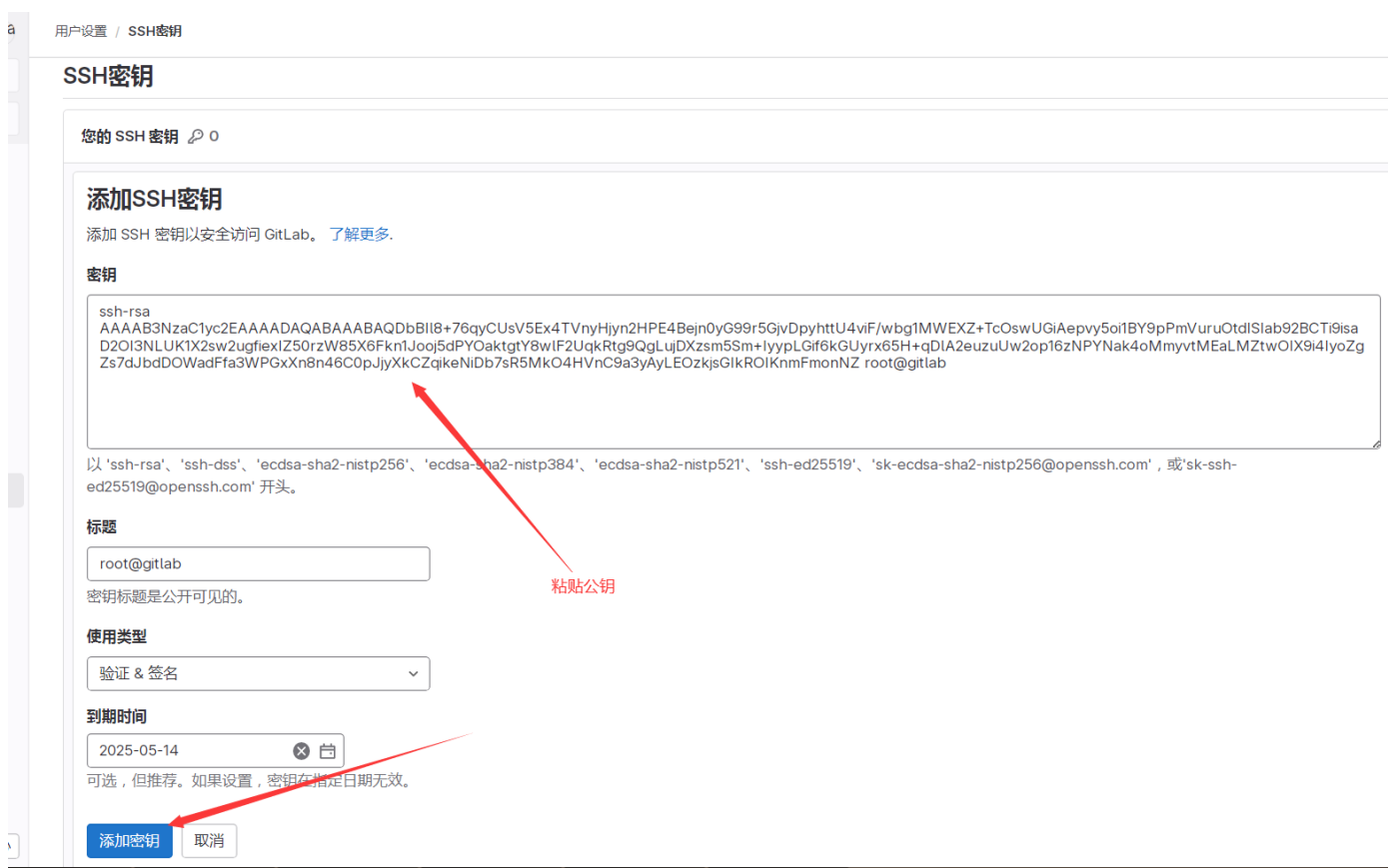
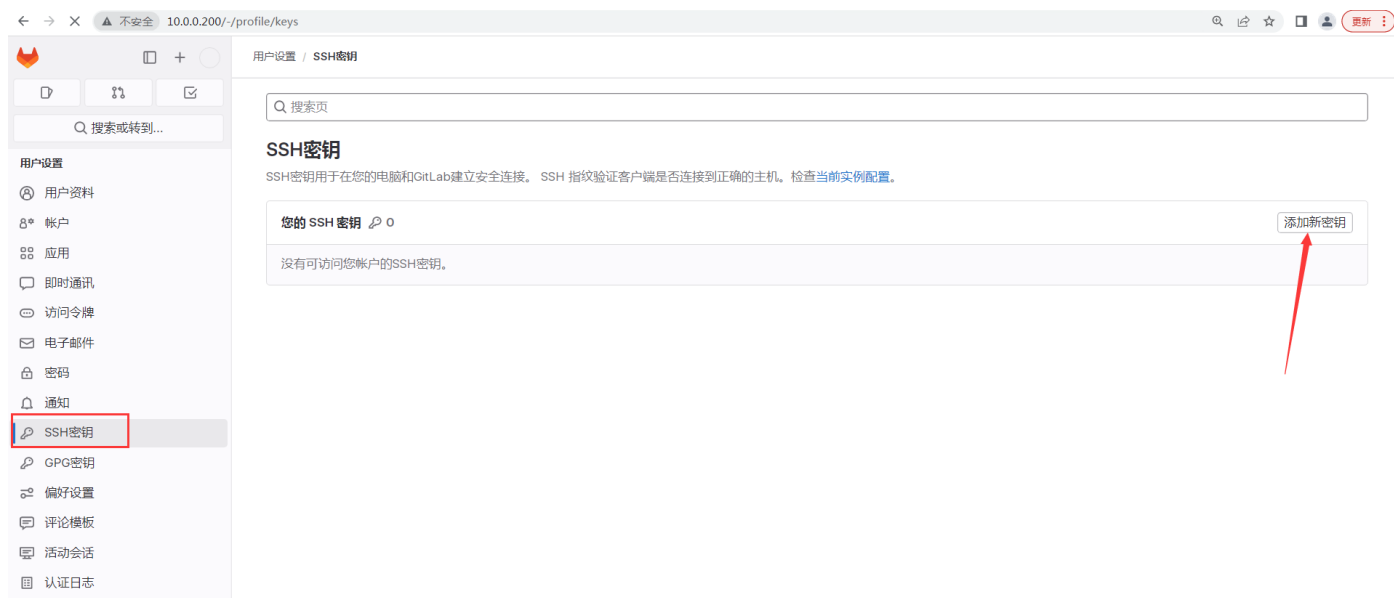
```
20 | ++o =. . |
21 | . o*o. |
22 +---[SHA256]-----+
23
24 查看秘钥对:
25 [root@gitlab ~]# ll .ssh/
26 total 8
27 -rw----- 1 root root 1679 May 14 15:02 id_rsa          # 私钥
28 -rw-r--r-- 1 root root 393 May 14 15:02 id_rsa.pub       # 公钥
29 [root@gitlab ~]# cat .ssh/id_rsa.pub
30 ssh-rsa
   AAAAB3NzaC1yc2EAAAADAQABAAQDbBI78+76qyCusV5Ex4TVnyHjyn2HPE4Bejn0yG99r5GjvDpyhttU4viF/wbg1MWEXZ+TcOswUGi
   Aepvy5oi1BY9pPmVuruOtdISIab92BCTi9isad2OI3NLUK1X2sw2ugfiexIZ50rzW85X6Fkn1Jooj5dPYOaktgtY8w1F2UqkRtg9QgLujD
   Xzsm5Sm+IyyPLGif6kGUyrx65H+qD1A2euzuUw2op16zNPYNak4oMmyvtMEaLMZtwOIX9i4IyoZgZs7dJbdDOWadFfa3wPGxXn8n46C0pJ
   jyXkCZqikeNidb7sR5MkO4Hvnc9a3yAyLEOzkjsGIkROIknmFmonNZ root@gitlab
31
32
```

2.将生成的公钥粘贴到gitlab页面的SSH秘钥管理页面

The screenshot shows the GitLab web interface. On the left is a sidebar with navigation links: 项目 (Projects), 已钉选 (Pinned), 议题 (Issues), 合并请求 (Merge Requests), 管理 (Settings), 计划 (Plan), 代码 (Code), 构建 (Builds), 安全 (Security), 部署 (Deployments), 运维 (Operations), 监控 (Monitoring), 分析 (Analytics), and 设置 (Settings). The main content area shows the 'Game' project page. At the top, there is a warning message: '在您的个人资料中添加SSH密钥之前，您不能通过SSH来拉取或推送仓库。' (Before adding an SSH key to your profile, you cannot pull or push to the repository via SSH). Below this is a button '添加SSH密钥' (Add SSH key) and a button '不再显示' (Don't show again). A red arrow points to the '添加SSH密钥' button. Below the warning is a success message: '项目 'Game' 已创建成功。' (Project 'Game' has been created successfully). The project name 'Game' is displayed with a lock icon. Below this is the section '邀请您的团队' (Invite your team) with a subtext '添加成员到此项目并开始与您的团队合作。' (Add members to this project and start collaborating with your team.) and a button '邀请成员' (Invite members). The next section is '该项目仓库当前为空' (This project's repository is currently empty) with a subtext '您可以通过克隆仓库开始或开始以以下方式之一添加文件。' (You can start by cloning the repository or start adding files in one of the following ways.). Below this are several buttons: '代码' (Code), '上传文件' (Upload file), '新建文件' (New file), '添加自述文件' (Add README), '添加LICENSE', '添加更新日志' (Add changelog), and '添加' (Add). The '代码' button is expanded, showing a dropdown menu. Below this is the '命令行指引' (Command line guide) section with a subtext '您还可以按照以下说明从计算机中上传现有文件。' (You can also follow the following instructions to upload existing files from your computer.). The 'Git 全局设置' (Git global settings) section shows the following commands:

```
git config --global user.name "Administrator"
git config --global user.email "admin@example.com"
```

 The final section is '创建一个新仓库' (Create a new repository).



至此就完成了服务器和gitlab页面之间的SSH免密钥,接下来将代码推送到game项目中

推送代码的方式有两种:

1.将空的仓库克隆到本地,然后将代码拷贝到仓库中,然后提交到远程仓库gitlab

进入到项目中找到项目的链接地址

邀请您的团队

添加成员到此项目并开始与您的团队合作。

[邀请成员](#)

该项目仓库当前为空

您可以通过克隆仓库开始或开始以以下方式之一添加文件。

代码

[上传文件](#)[新建文件](#)[添加自述文件](#)[添加LICENSE](#)[添加更新日志](#)[添加贡献信息](#)[添加 Wiki](#)

使用 SSH 克隆



使用 HTTP 克隆



在您的IDE中打开

Visual Studio Code (SSH)

Visual Studio Code (HTTPS)

IntelliJ IDEA (SSH)

IntelliJ IDEA (HTTPS)

```
git commit -m "add README"
git push --set-upstream origin main
```

```
1 配置使用服务器的人员姓名和邮箱地址：
2 [root@gitlab game]# git config --global user.email "dev@qq.com"
3 [root@gitlab game]# git config --global user.name "dev"
4
5 1.复制链接执行
6 [root@gitlab ~]# git clone git@10.0.0.200:ops/game.git
7 Cloning into 'game'...
8 The authenticity of host '10.0.0.200 (10.0.0.200)' can't be established.
9 ECDSA key fingerprint is SHA256:RMA62gxm552WQwQmP5CpXl4EaToCw9+rjp0i8/Bg9e4.
10 ECDSA key fingerprint is MD5:79:a1:37:91:d1:15:40:79:f2:d3:11:fd:e8:ca:4d:23.
11 Are you sure you want to continue connecting (yes/no)? yes
12 Warning: Permanently added '10.0.0.200' (ECDSA) to the list of known hosts.
13 warning: remote HEAD refers to nonexistent ref, unable to checkout.
14
15 2.查看将game仓库下载到了本地
16 [root@gitlab ~]# ll
17 total 0
18 drwxr-xr-x 3 root root 18 May 14 15:18 game
19
20 3.进入game目录上传游戏代码
21 [root@gitlab ~]# cd game/
22 [root@gitlab game]# rz -E
23 rz waiting to receive.
24 [root@gitlab game]# ll
25 total 7720
26 -rw-r--r-- 1 root root 7902976 May 13 08:34 xbw.zip
27
28
29 4.解压代码：
30 [root@gitlab game]# unzip xbw.zip
31
32 5.将代码提交到本地仓库
33 [root@gitlab game]# git add .
34 [root@gitlab game]# git commit -m "game_v1"
35
36 6.将代码提交到远程仓库gitlab的game仓库中
37 因为我们是直接克隆的gitlab的仓库下载到本地的所以目录中默认已经有了远程仓库game的项目地址
38 查看远程仓库：
```

```

39 [root@gitlab game]# git remote -v
40 origin git@10.0.0.200:ops/game.git (fetch)
41 origin git@10.0.0.200:ops/game.git (push)
42
43 将本地的master推送到远程仓库origin
44 [root@gitlab game]# git push -u origin master
45 Counting objects: 101, done.
46 Compressing objects: 100% (98/98), done.
47 Writing objects: 100% (100/100), 7.35 MiB | 3.86 MiB/s, done.
48 Total 100 (delta 4), reused 0 (delta 0)
49 remote: Resolving deltas: 100% (4/4), done.
50 To git@10.0.0.200:ops/game.git
51    d24a5f9..bde4b0a  master -> master
52 Branch master set up to track remote branch master from origin.
53

```

查看game项目,代码上传成功

ops游戏组 / Game

G Game

🔗 派生 0

🔔 通知

🔖 星标 0

🔗 链接

📁 1 次提交

🌿 1 个分支

🏷️ 0 个标签

💾 45 KB 项目存储

xxx dev 编辑于 2分钟后

bde4b0ad

master game / +

历史 查找文件 编辑 代码

🔧 启用Auto DevOps

📄 添加自述文件

📄 添加LICENSE

📄 添加更新日志

📄 添加贡献信息

📄 添加 Kubernetes 集群

📄 添加 Wiki

⚙️ 配置集成

名称	最后提交	最后更新
css	xx	2分钟后
images	xx	2分钟后
js	xx	2分钟后
roms	xx	2分钟后
bgm.mp3	xx	2分钟后
index.html	xx	2分钟后
shuoming.html	xx	2分钟后

2.将本地已有的git仓库推送到远程仓库gitlab

```

1 1.配置一个已存在的本地仓库
2 [root@gitlab ~]# mkdir git_data
3 [root@gitlab ~]# cd git_data/
4 2.初始化仓库
5 [root@gitlab git_data]# git init
6 3.上传或者已存在的代码
7 [root@gitlab git_data]# touch a.txt
8 [root@gitlab git_data]# git add.
9 [root@gitlab git_data]# git commit -m "test_v1"
10 4.设置gitlab的game仓库为本地的远程仓库
11 [root@gitlab git_data]# git add remote origin git@10.0.0.200:ops/game.git
12 5.将本地仓库的内容提交到远程仓库
13 [root@gitlab git_data]# git push -u origin master

```

3.创建普通账号dev

🔍 搜索或转到...

管理中心

📁 概览

仪表盘

项目

用户

群组

主题

Gitaly服务器

🔧 CI/CD

📊 分析

📺 监控

📢 消息

🔗 系统钩子

📦 应用程序

🚫 滥用报告

🔑 部署密钥

🏷️ 标记

⚙️ 设置

🔍 帮助

🔑 管理中心

新用户

帐户

名称

dev

用户名

dev

电子邮件

dev@qq.com

密码

🔔 将生成重置链接并发送给用户。用户将被迫在首次登录时设置密码。

访问类型

Projects limit

100000

☒ 可以创建顶级群组

☐ 个人资料

访问级别

☒ 普通

普通用户可以访问他们的群组和项目。

☐ 管理员

用户可以无限制地访问所有群组、项目、用户和功能。

创建用户

取消

创建完成后修改dev密码-别退出，点击用户界面的编辑



□

+



📄

🔗

✉️

🔍 搜索或转到...

管理中心

📑 概览

仪表盘

项目

用户

群组

主题

GitLab 服务器

🔧 CI/CD

📊 分析

📺 监控

📢 消息

🔗 系统钩子

🧩 应用程序

🚨 滥用报告

🔑 部署密钥

🏷️ 标记

管理中心 / 用户

编辑用户: dev

帐户

名称

dev

用户名

dev

电子邮件

dev@qq.com

密码

密码

密码确认

访问类型

Projects limit

100000

完成后使用dev登录gitlab(第一次登录需要修改密码)

4.将dev用户添加到oldboy组



□

+



📄

🔗

✉️

🔍 搜索或转到...

管理中心

📑 概览

仪表盘

项目

用户

群组

主题

GitLab 服务器

🔧 CI/CD

📊 分析

📺 监控

📢 消息

🔗 系统钩子

🧩 应用程序

🚨 滥用报告

🔑 部署密钥

🏷️ 标记

🔧 设置

📖 帮助

管理中心 / 群组 / oldboy

群组: oldboy

🔗 编辑

群组信息

🔗

名称: oldboy

路径: oldboy

描述:

可见性级别: 私有

创建于: Jun 11, 2024 8:26am

ID: 2

存储: 7.8 MB (仓库: 7.8 MB / Wikis: 0 字节 / 构建产物: 0 字节 / 流水线产物: 0 字节 / LFS: 0 字节 / 代码片段: 0 字节 / 软件包: 0 字节 / 上传文件: 0 字节)

群组Git LFS状态: Enabled for all projects

项目 1

oldboy / game 7.8 MB oldboy/game.git

oldboy 群组成员 2

 Administrator @root 你自己 49分钟前授权访问 3

Owner

 dev @dev 15分钟前授权访问

Developer

🔗 管理权限


```

5 [root@1b01 game]#vim index.html
6 xxx
7 [root@1b01 game]#git commit -am "魂斗罗"
8 提交到远程仓库
9 [root@1b01 game]#git push -u origin master # 不允许推送到master分支
10
11 推送到其他分支:
12 1.创建dev分支
13 [root@1b01 game]#git branch dev
14 [root@1b01 game]#git branch
15     dev
16 * master
17 [root@1b01
18
19 2.推送dev分支到远程仓库
20 [root@1b01 game]#git push -u origin dev
21
22

```

Jenkins服务

01. Jenkins简介

1. 官网 jenkins.io
2. Jenkins是一个开源软件项目，是基于Java开发的一种持续集成工具，用于监控持续重复的工作，旨在提供一个开放易用的软件平台，使软件的持续集成变成可能。

02. Jenkins部署

```

1 一.环境要求
2 1、CentOS7
3 2、2G内存（实验）生产（至少6G）
4 3、安装包: gitlab-ce-16.9.8-ce
5 4、禁用防火墙，关闭selinux
6 5、JDK17
7
8 二.安装JDK
9 上传rpm包
10 [root@jenkins ~]# ll
11 total 178212
12 -rw-r--r-- 1 root root 182486159 Mar 12 02:06 jdk-17_linux-x64_bin.rpm
13
14 [root@jenkins ~]# rpm -ivh jdk-17_linux-x64_bin.rpm
15 warning: jdk-17_linux-x64_bin.rpm: Header V3 RSA/SHA256 Signature, key ID ad986da3: NOKEY
16 Preparing... ##### [100%]
17 Updating / installing...
18 1:jdk-17-2000:17.0.11-7 ##### [100%]
19
20 查看是否安装成功:
21 [root@jenkins ~]# rpm -qa|grep jdk
22 jdk-17-17.0.11-7.x86_64
23
24 二.安装jenkins
25 上传jenkins包
26 [root@jenkins ~]# ll
27 total 269432
28 -rw-r--r-- 1 root root 182486159 Mar 12 02:06 jdk-17_linux-x64_bin.rpm
29 -rw-r--r-- 1 root root 93405530 May 16 2023 jenkins-2.405-1.1.noarch.rpm
30 [root@jenkins ~]# rpm -ivh jenkins-2.405-1.1.noarch.rpm

```

```
31 warning: jenkins-2.405-1.1.noarch.rpm: Header V4 RSA/SHA512 Signature, key ID ef5975ca: NOKEY
32 Preparing... ##### [100%]
33 Updating / installing...
34 1:jenkins-2.405-1.1 ##### [100%]
35
36 三.修改Jenkins启动用户为root
37 1)修改配置文件的启动用户
38 [root@jenkins ~]# vim /etc/sysconfig/jenkins
39 ...
40 JENKINS_USER="root"
41 ...
42 2)修改启动配置的用户
43 [root@jenkins ~]# vim /usr/lib/systemd/system/jenkins.service
44 ...
45 User=root
46 Group=root
47 ...
48 [root@jenkins ~]# systemctl daemon-reload
49
50 3)安装字体配置
51 [root@jenkins ~]# yum -y install fontconfig
52
53 四.启动Jenkins并加入开机自动启动
54 [root@jenkins ~]# systemctl start jenkins
55 [root@jenkins ~]# systemctl enable jenkins
```

03. 登陆Jenkins

← → ↻ ▲ 不安全 | 10.0.0.201:8080/login?from=%2F

入门

解锁 Jenkins

为了确保管理员安全地安装 Jenkins，密码已写入到日志中（[不知道在哪里？](#)）该文件在服务器上：

```
/var/lib/jenkins/secrets/initialAdminPassword
```

请从本地复制密码并粘贴到下面。

管理员密码

存放密码的位置,进入后记得修改默认密码

继续

自定义Jenkins

插件通过附加特性来扩展Jenkins以满足不同的需求。

跳过安装插件步骤

安装推荐的插件

安装Jenkins社区推荐的插件。

选择插件来安装

选择并安装最适合的插件。

Jenkins已就绪！

你已跳过创建admin用户的步骤。要登录请使用用户名：'admin' 及用于访问安装向导的管理员密码。

您已经跳过了 Jenkins URL的配置。

要配置 Jenkins URL的话，到“Jenkins管理”页面。

Jenkins安装已完成。

开始使用Jenkins

04. 修改密码

修改完密码后会重新登陆,用户名为admin

Builds

Configure

My Views

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job



Set up a distributed build

Set up an agent



Dashboard > admin > Configure

Password

Password:

.....

Confirm Password:

.....

Session Termination

Terminate All Sessions



Setting for search

Save

Apply

05. 插件安装

```
1 1.上传插件到jenkins家目录
2 [root@jenkins ~]# cd /var/lib/jenkins/
3 [root@jenkins jenkins]# rz -bye
4 rz waiting to receive.
5 Starting zmodem transfer. Press Ctrl+C to cancel.
6 Transferring jenkins_plugins.tar.gz...
7 100% 253838 KB 8461 KB/sec 00:00:30 0 Errors
8
9 [root@jenkins jenkins]# ll jenkins_plugins.tar.gz
10 -rw-r--r-- 1 root root 259930924 May 14 10:28 jenkins_plugins.tar.gz
11 2.将插件解压到plugins目录中
12 [root@jenkins jenkins]# tar xf jenkins_plugins.tar.gz -C plugins/
13
14 3.重启Jenkins生效
15 [root@jenkins ~]# systemctl restart jenkins
```

06. 创建项目

创建一个自由风格项目,执行pwd命令,查看使用jenkins运行默认的路径所在位置

Dashboard > 所有 >

输入一个任务名称

test_job

» 必填项

选择自由风格



构建一个自由风格的软件项目

这是Jenkins的主要功能.Jenkins将会结合任何SCM和任何构建系统来构建你的项目,甚至可以



构建一个maven项目

构建一个maven项目.Jenkins利用你的POM文件,这样可以大大减轻构建配置.



流水线

精心地组织一个可以长期运行在多个节点上的任务。适用于构建流水线（更加正式地应当称为。的任务类型。



构建一个多配置项目

适用于多配置项目,例如多环境测试,平台指定构建,等等.

确定

文件夹

is a set of multibranch project subfolders by scanning for repositories.

Configure

General

源码管理

构建触发器

构建环境

Build Steps

构建后操作

- ☐ Add Changelog Information to Environment
- ☐ Inspect build log for published build scans
- ☐ Terminate a build if it's stuck
- ☐ With Ant ?
- ☐ 在构建日志中添加时间戳前缀

Build Steps

增加构建步骤 ^

Filter

- Execute SonarQube Scanner
- Invoke Ant
- Invoke Gradle script
- Run with timeout
- Set build status to "pending" on GitHub commit
- SonarScanner for MSBuild - Begin Analysis
- SonarScanner for MSBuild - End Analysis
- 执行 Windows 批处理命令
- 执行 shell
- 调用顶层 Maven 目标

点击执行shell

源码管理

构建触发器

构建环境

Build Steps

构建后操作

☐ With Ant ?

☐ 在构建日志中添加时间戳前缀

Build Steps

执行 shell ?

命令

查看 可用的环境变量列表

pwd

保存

应用

☰ 状态

</> 修改记录

📁 工作空间

▶ 立即构建

⚙️ 配置

🗑️ 删除 工程

✎ 重命名

工程 test_job

相关链接

点击立即构建

☀️ 构建历史

趋势 ▼

No builds

☰ 状态

</> 修改记录

📁 工作空间

▶ 立即构建

⚙️ 配置

🗑️ 删除 工程

✎ 重命名

工程 test_job

相关链接

- 最近一次构建(#1),12 秒之前
- 最近稳定构建(#1),12 秒之前
- 最近成功的构建(#1),12 秒之前
- 最近完成的构建(#1),12 秒之前

☀️ 构建历史

Q 过滤构建...

✓ #1
| 2024年5月15日 上午8:46

📡 Atom feed 全部 📡 Atom feed 失败

</> 变更记录

📄 控制台输出

☑ 编辑编译信息

🗑️ 删除构建 '#1'

完成后会显示BUILD构建结果,点击小箭头选择控制台输出查看执行结果

状态集

</> 变更记录

控制台输出

文本方式查看

编辑编译信息

删除构建 ' #1 '

 控制台输出

```
Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/test_job
[test_job] $ /bin/sh -xe /tmp/jenkins13726251072616580498.sh
+ pwd
/var/lib/jenkins/workspace/test_job
Finished: SUCCESS
```

执行Shell命令默认所在的路径

说明: 使用jenkins创建完项目后,系统会自动在jenkins的家目录/var/lib/jenkins/下创建一个workspace项目目录, 并且在此目录下创建以项目名称命名的目录。

后续jenkins拉取gitlab的代码就是存储在此项目目录下。

```
1 [root@jenkins ~]# ll /var/lib/jenkins/workspace/
2 total 0
3 drwxr-xr-x 2 root root 6 May 15 08:46 test_job
```

SonarQube服务

01. SonarQube简介

1. SonarQube基础java开发, 需安装open JDK8版本
2. SonarQube需要依赖MySQL数据库, 至少5.6版本以上
3. SonarQube的小型实例至少4G内存, 如果大型实例需要16G内存

02. 部署服务

一. 安装运行环境JDK

```
1 [root@sonar ~]# ll
2 total 166052
3 -rw-r--r-- 1 root root 170023183 Aug 14 2018 jdk-8u181-linux-x64.rpm
4 [root@sonar ~]# rpm -ivh jdk-8u181-linux-x64.rpm
5 warning: jdk-8u181-linux-x64.rpm: Header V3 RSA/SHA256 Signature, key ID ec551f03: NOKEY
6 Preparing... ##### [100%]
7 Updating / installing...
8 1:jdk1.8-2000:1.8.0_181-fcs ##### [100%]
```

二. 安装数据库

```
1 # 1. 下载MySQL官方的Yum仓库配置文件
2 [root@sonar ~]# wget dev.mysql.com/get/mysql-community-release-el6-5.noarch.rpm
3
4 # 2. 安装MySQL源
5 [root@sonar ~]# rpm -ivh dev.mysql.com/get/mysql-community-release-el6-5.noarch.rpm
6
7 # 3. 安装MySQL服务器
8 [root@sonar ~]# yum install mysql-community-server -y
9
10 # 4. 启动MySQL服务
11 [root@sonar ~]# service mysqld start
12
```

```

13 # 5. 设置密码
14 [root@sonar ~]# mysqladmin -uroot password lizhenya123
15
16 # 6. 创建sonar库
17 [root@sonar ~]# mysql -uroot -plizhenya123 -e "CREATE DATABASE sonar DEFAULT CHARACTER SET utf8;"
18
19 [root@sonar ~]# mysql -u root -plizhenya123 -e "show databases"
20 Warning: Using a password on the command line interface can be insecure.
21 +-----+
22 | Database                |
23 +-----+
24 | information_schema      |
25 | mysql                   |
26 | performance_schema      |
27 | sonar                   |
28 +-----+
29

```

三. 安装sonarqube

```

1 # 1. 上传soanrqube
2 [root@sonar ~]# cd /usr/local/
3 [root@sonar local]# ll sonarqube-7.0.zip
4 -rw-r--r-- 1 root root 155709573 Oct 23 2019 sonarqube-7.0.zip
5
6 # 2. 解压到当前目录
7 [root@sonar local]# unzip sonarqube-7.0.zip
8
9 [root@sonar local]# ln -s sonarqube-7.0 sonarqube
10
11 # 3. 安装插件:
12 删除默认的插件:
13 [root@sonar ~]# rm -rf /usr/local/sonarqube/extensions/plugins/*
14
15 上传插件:
16 [root@sonar ~]# ll sonar_plugins.tar.gz
17 -rw-r--r-- 1 root root 45106788 Oct 24 2019 sonar_plugins.tar.gz
18
19 [root@sonar ~]# tar xf sonar_plugins.tar.gz -C /usr/local/sonarqube/extensions/plugins/
20 [root@sonar ~]# ll /usr/local/sonarqube/extensions/plugins/
21 total 4
22 drwxr-xr-x 2 sonar sonar 4096 Oct 24 2019 plugins
23 [root@sonar ~]# cd /usr/local/sonarqube/extensions/plugins/
24 [root@sonar plugins]# mv plugins/* .
25
26
27 # 4. 创建普通用户
28 [root@sonar ~]# useradd sonar
29
30 # 5. 授权代码属主属组为sonar
31 [root@sonar ~]# chown -R sonar.sonar /usr/local/sonarqube/
32 [root@sonar ~]# ll /usr/local/sonarqube/
33 total 12
34 drwxr-xr-x 8 sonar sonar 136 Feb 2 2018 bin
35 drwxr-xr-x 2 sonar sonar 50 May 15 15:09 conf
36 -rw-r--r-- 1 sonar sonar 7651 Feb 2 2018 COPYING
37 drwxr-xr-x 2 sonar sonar 24 Feb 2 2018 data
38 drwxr-xr-x 7 sonar sonar 150 Feb 2 2018 elasticsearch
39 drwxr-xr-x 4 sonar sonar 40 Feb 2 2018 extensions
40 drwxr-xr-x 9 sonar sonar 138 Feb 2 2018 lib
41 drwxr-xr-x 2 sonar sonar 6 Feb 2 2018 logs
42 drwxr-xr-x 2 sonar sonar 24 Feb 2 2018 temp
43 drwxr-xr-x 9 sonar sonar 4096 Feb 2 2018 web
44
45

```

```

46 # 6. 配置连接数据库
47
48 [root@sonar ~]# vim /usr/local/sonarqube/conf/sonar.properties
49 ...省略
50 sonar.jdbc.username=root          # 数据库用户名
51 sonar.jdbc.password=lizhenya123   # 数据库密码
52
53 #----- Embedded Database (default)
54 # H2 embedded database server listening port, defaults to 9092
55 #sonar.embeddedDatabase.port=9092
56
57 #----- MySQL 5.6 or greater
58 # Only InnoDB storage engine is supported (not myISAM).
59 # Only the bundled driver is supported. It can not be changed.
60 sonar.jdbc.url=jdbc:mysql://localhost:3306/sonar?useUnicode=true&characterE
61 ncoding=utf8&rewriteBatchedStatements=true&useConfigs=maxPerformance&useSSL
62 =false          # 开启本地连接
63 ...省略

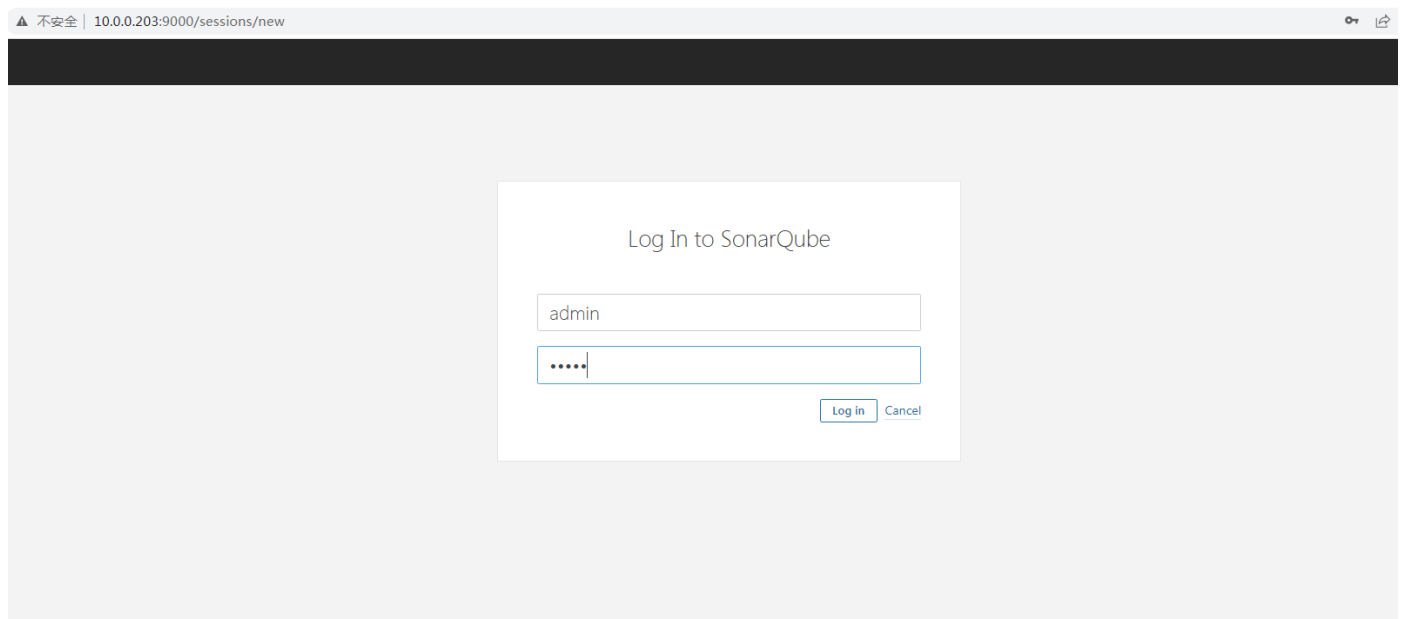
```

四. 运行SonarQube

- 1 | SonarQube服务启动不能使用root, 必须使用普通用户运行
- 2 | [root@sonar ~]# su - sonar -c "/usr/local/sonarqube/bin/linux-x86-64/sonar.sh start"

五. 配置远程访问token

启动后的端口为9000使用浏览器直接访问<http://10.0.0.203:9000/about> 用户名admin 密码admin 生成一个token,只显示一次记录好。



Welcome to SonarQube!

Want to quickly analyze a first project? Follow these 2 easy steps.

1 Provide a token

html: a08b950a03a662c7fe91d89289794881622388ee ✖

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point of time in your user account.

Continue

2 Run analysis on your project

Welcome to SonarQube!

Want to quickly analyze a first project? Follow these 2 easy steps.

1 Provide a token

✔ html: a

2 Run analysis on your project

What is your project's main language?

Java

C# or VB.NET

Other (JS, Python, PHP, ...)

What is your OS?

Linux

Windows

macOS

Define a unique project key

html

Done

Allowed characters are alphanumeric, '-', '_', '.' and ':', with at least one non-digit. 400 characters max.

1 Provide a token

✔ html: a08b950a03a662c7fe91d89289794881622388ee

2 Run analysis on your project

What is your project's main language?

Java

C# or VB.NET

Other (JS, Python, PHP, ...)

What is your OS?

Linux

Windows

macOS

Define a unique project key

html ✖

Download and unzip the SonarQube Scanner for Linux

And add the `bin` directory to the `PATH` environment variable

Download

复制保留

Execute the SonarQube Scanner from your computer

Running a SonarQube analysis is straightforward. You just need to execute the following commands in your project's folder.

```
sonar-scanner \
-Dsonar.projectKey=html \
-Dsonar.sources=. \
-Dsonar.host.url=http://10.0.0.203:9000 \
-Dsonar.login=a08b950a03a662c7fe91d89289794881622388ee
```

Copy

Please visit the [official documentation of the SonarQube Scanner](#) for more details.

Once your project is analyzed, this page will refresh automatically.

```
1 [root@jenkins ~]# sonar-scanner -Dsonar.projectKey=html -Dsonar.sources=. -  
Dsonar.host.url=http://10.0.0.203:9000 -Dsonar.login=a08b950a03a662c7fe91d89289794881622388ee
```

六. 客户端部署

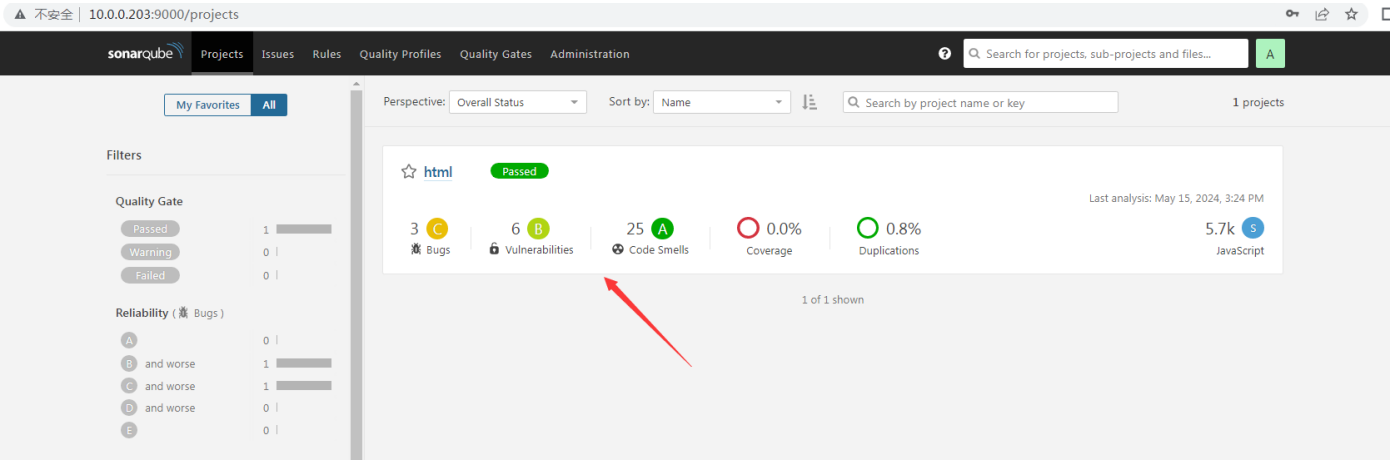
客户端IP:10.0.0.201

```
1 # 1. 上传客户端  
2 [root@jenkins ~]# cd /usr/local/  
3 [root@jenkins local]# ll sonar-scanner-cli-4.2.0.1873-linux.zip  
4 -rw-r--r-- 1 root root 42397119 Oct 1 2019 sonar-scanner-cli-4.2.0.1873-linux.zip  
5  
6 # 2. 解压做软链接  
7 [root@jenkins local]# unzip sonar-scanner-cli-4.2.0.1873-linux.zip  
8 [root@jenkins local]# ln -s sonar-scanner-4.2.0.1873-linux sonar  
9  
10 # 3. 设置PATH全局变量  
11 [root@jenkins ~]# export PATH="$PATH:/usr/local/sonar/bin/"  
12  
13 # 4. 永久变量写入/etc/profile  
14 [root@jenkins ~]# vim /etc/profile  
15 ..  
16 export PATH="$PATH:/usr/local/sonar/bin/"  
17  
18 # 5. 客户端测试将代码扫描到服务器进行代码检测  
19 将game目录下的游戏代码推送到SonrQube测试  
20 [root@jenkins ~]# cd /var/lib/jenkins/workspace/game_job/  
21 [root@jenkins game_job]# ll  
22 total 48  
23 -rw-r--r-- 1 root root 28032 May 15 09:16 bgm.mp3  
24 drwxr-xr-x 2 root root 23 May 15 09:16 css  
25 drwxr-xr-x 2 root root 23 May 15 09:16 images  
26 -rw-r--r-- 1 root root 8956 May 15 09:16 index.html  
27 drwxr-xr-x 2 root root 213 May 15 09:16 js  
28 drwxr-xr-x 2 root root 4096 May 15 09:16 roms  
29 -rw-r--r-- 1 root root 811 May 15 09:16 shuoming.html  
30 [root@jenkins game_job]# sonar-scanner \  
31 > -Dsonar.projectKey=html \  
32 > -Dsonar.sources=. \  
33 > -Dsonar.host.url=http://10.0.0.203:9000 \  
34 > -Dsonar.login=a08b950a03a662c7fe91d89289794881622388ee
```

执行成功:

```
k?id=AY97I8RSFm2j06RIjRXw  
INFO: Task total time: 18.604 s  
INFO: -----  
---  
INFO: EXECUTION SUCCESS  
INFO: -----  
---  
INFO: Total time: 26.644s  
INFO: Final Memory: 13M/144M  
INFO: -----  
---
```

查看服务器: 上传成功



Jenkins项目实战

01. Jenkins拉取gitlab代码

一. gitlab创建仓库上传游戏代码

GGame

3 次提交 1 个分支 0 个标签 7.4 MB 项目存储

xxx dev 编辑于 17小时前 bde4b0ad

master game + 历史 查找文件 编辑 代码

启用Auto DevOps 添加自述文件 添加LICENSE 添加更新日志 添加贡献信息 添加 Kubernetes 集群 添加 Wiki 配置集成

名称	最后提交	最后更新
css	xx	17小时前
images	xx	17小时前
js	xx	17小时前
roms	xx	17小时前
bgm.mp3	xx	17小时前
index.html	...	17小时前

二. Jenkins创建项目

1.创建项目

输入一个任务名称

game_job

» 必填项



构建一个自由风格的软件项目

这是Jenkins的主要功能.Jenkins将会结合任何SCM和任何构建系统来构建你的项目, 甚至可以构建!



构建一个maven项目

构建一个maven项目.Jenkins利用你的POM文件,这样可以大大减轻构建配置.



流水线

精心地组织一个可以长期运行在多个节点上的任务。适用于构建流水线（更加正式地应当称为工作的任务类型。



构建一个多配置项目

适用于多配置项目,例如多环境测试,平台指定构建,等等.

确定

set of multibranch project subfolders by scanning for repositories.

2.配置连接的gitlab项目地址

Configure

General

源码管理

构建触发器

构建环境

Build Steps

构建后操作

源码管理

☐ 无☒ Git ?

Repositories ?

Repository URL ?

git@10.0.0.200:ops/game.git

无法连接仓库: Command "git ls-remote -h git@10.0.0.200:ops/game.git HEAD" returned status code 128:
stdout:
stderr: Host key verification failed.
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

Credentials ?

- 无 -

添加

gitlab游戏代码项目地址

提示host key验证失败,需要将Jenkins的公钥复制到gitlab做免秘钥配置

3.Jenkins服务器生成密钥对

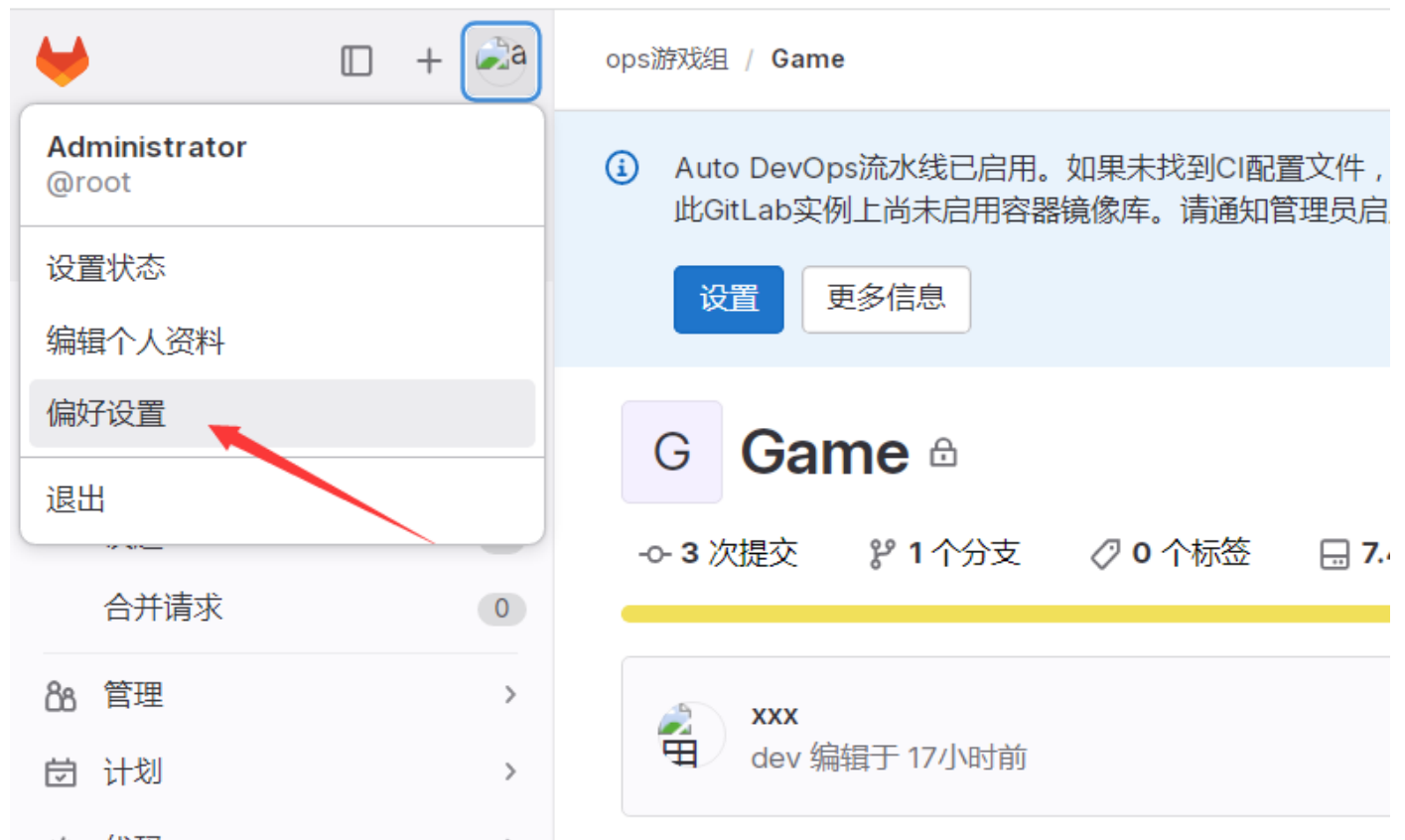
```
1 [root@jenkins ~]# ssh-keygen # 一路回车
2 Generating public/private rsa key pair.
3 Enter file in which to save the key (/root/.ssh/id_rsa):
4 Enter passphrase (empty for no passphrase):
5 Enter same passphrase again:
```

```

6 Your identification has been saved in /root/.ssh/id_rsa.
7 Your public key has been saved in /root/.ssh/id_rsa.pub.
8 The key fingerprint is:
9 SHA256:Pufpx2Ei8INIGvKqxsV2AKYad1DVM2hgycAE101ZDQY root@jenkins
10 The key's randomart image is:
11 +---[RSA 2048]-----+
12 |. +*BE***. |
13 |o . =B. =+ . |
14 |o + . * = . . o + + o |
15 |. = *o . = + . |
16 |. + S o o o |
17 | o o . |
18 |. |
19 | |
20 | |
21 +---[SHA256]-----+

```

4.将公钥拷贝到gitlab页面的SSH密钥验证页面



The screenshot shows the GitLab web interface. On the left, a user menu is open for the user 'Administrator @root'. The menu options are: '设置状态' (Set status), '编辑个人资料' (Edit profile), '偏好设置' (Preferences), and '退出' (Logout). A red arrow points to the '偏好设置' option. Below the menu, there are links for '合并请求' (Merge requests) with a count of 0, '管理' (Manage), and '计划' (Plan). The main content area shows the repository 'ops游戏组 / Game'. A notification banner states: 'Auto DevOps流水线已启用。如果未找到CI配置文件，此GitLab实例上尚未启用容器镜像库。请通知管理员启' (Auto DevOps pipeline is enabled. If no CI configuration file is found, container image repository is not yet enabled on this GitLab instance. Please notify the administrator to enable). Below the notification are buttons for '设置' (Settings) and '更多信息' (More information). The repository details show '3 次提交' (3 commits), '1 个分支' (1 branch), '0 个标签' (0 tags), and a version '7.'. A commit entry is visible, showing a user 'xxx' with the alias 'dev' who edited the file 17 hours ago.

← → × 不安全 | 10.0.0.200 /profile/keys

用户设置 / SSH密钥

搜索页

SSH密钥

SSH密钥用于在您的电脑和GitLab建立安全连接。SSH 指纹验证客户端是否连接到正确的主机。检查[当前实例配置](#)。

您的 SSH 密钥 1

标题	密钥	使用类型	已创建	最近使用	过期	操作
root@gitlab	22:34:83:3c:f5:73:67:fe:a1:9a:ec:66:d3:47:17:95	验证 & 签名	17小时前	17小时前	2025-05-14	撤销 删除

[添加新密钥](#)

SSH密钥

您的 SSH 密钥 1

添加SSH密钥

添加 SSH 密钥以安全访问 GitLab。 [了解更多](#)。

密钥

```
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQDpZFRaCdCY7J1I5VIPVLEXk4kANQD36/Z+Zrm5dRCO+Y3Qe8IWSXIUExwkMEpU2H6lxGqzbu6pXD5WZutdoTNrN4paQNHg+GFevXbP8
ASRzuQLna8zZU19HIqDiWCXr4jExGm4/3q+FZvdELYLHx/MIlpOjdMmY235N7BaDtpsXI4f4wA44KpC6oNwd7+jl/l44euavpk91l+v8cNoGT19GJNz9SkvGUyWl6O0q8EOo3SCBUk+GTN
6SFo6GD4Wg9FdiBvZW6xHSK5mzcIkxllC+vjlndiKex1yb62cuHe3xxom2048SmAVLoY3on9rDbFkcQy9xqoPgZehEeR root@jenkins
```

以 'ssh-rsa', 'ssh-dss', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', 或 'sk-ssh-ed25519@openssh.com' 开头。

标题

root@jenkins

密钥标题是公开可见的。

使用类型

验证 & 签名

到期时间

2025-05-15

可选，但推荐。如果设置，密钥在指定日期无效。

[添加密钥](#) [取消](#)

帮助 管理中心

执行完成后还需要在jenkins服务器中执行一次克隆命令输入一次yes

```
10.0.0.201-jenkins x
[root@jenkins ~]# cd /tmp/
[root@jenkins tmp]# git clone git@10.0.0.200:ops/game.git
Cloning into 'game'...
The authenticity of host '10.0.0.200 (10.0.0.200)' can't be established.
ECDSA key fingerprint is SHA256:RMA62gXm552WQwQmP5CpXl4EaToCw9+rjpOi8/Bg9e4
.
ECDSA key fingerprint is MD5:79:a1:37:91:d1:15:40:79:f2:d3:11:fd:e8:ca:4d:2
3.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.0.0.200' (ECDSA) to the list of known hosts.
remote: Enumerating objects: 117, done.
remote: Counting objects: 100% (117/117), done.
remote: Compressing objects: 100% (108/108), done.
remote: Total 117 (delta 5), reused 100 (delta 4), pack-reused 0
Receiving objects: 100% (117/117), 7.39 MiB | 13.40 MiB/s, done.
Resolving deltas: 100% (5/5), done.
[root@jenkins tmp]#
[root@jenkins tmp]#
```

5.在查看jenkins的项目配置(如果还是看到报错提示,点击保存退出后在重新进入到项目)

Configure

- General
- 源码管理**
- 构建触发器
- 构建环境
- Build Steps
- 构建后操作

源码管理

☐ 无

☒ Git ?

Repositories ?

Repository URL ?

git@10.0.0.200:ops/game.git

Credentials ?

- 无 -

+ 添加

高级

显示正常,没有红色报错

完成后点击立即构建:

Dashboard > game_job >

状态

修改记录

工作空间

立即构建

配置

删除 工程

重命名

工程 game_job

相关链接

构建历史

趋势

No builds

Atom feed 全部

Atom feed 失败

📄 状态

</> 修改记录

📁 工作空间

▶ 立即构建

⚙️ 配置

🗑️ 删除 工程

✏️ 重命名

工程 game_job

相关链接

☀️ 构建历史 趋势 ▾

🔍 过滤构建... /

✅ #1 2024年5月15日 上午9:16

📡 Atom feed 全部 📡 Atom feed 失败

绿色显示成功,红色显示失败

查看项目目录下是否拉取游戏代码成功

```
1 [root@jenkins ~]# ll /var/lib/jenkins/workspace/game_job/
2 total 48
3 -rw-r--r-- 1 root root 28032 May 15 09:16 bgm.mp3
4 drwxr-xr-x 2 root root 23 May 15 09:16 css
5 drwxr-xr-x 2 root root 23 May 15 09:16 images
6 -rw-r--r-- 1 root root 8956 May 15 09:16 index.html
7 drwxr-xr-x 2 root root 213 May 15 09:16 js
8 drwxr-xr-x 2 root root 4096 May 15 09:16 roms
9 -rw-r--r-- 1 root root 811 May 15 09:16 shuoming.html
```

02. Jenkins推送代码到WEB

将拉取到的代码推送到WEB服务器的Nginx站点目录运行

```
1 1.创建WEB01服务器
2 2.安装Nginx服务
3 3.配置Nginx server
4 [root@web01 conf.d]# cat default.conf
5 server {
6     listen      80;
7     server_name localhost;
8
9
10    location / {
11        root    /code/html;
12        index  index.html index.htm;
13    }
14 }
```



```
15
16 4.jenkins和web服务器做免密钥
17 [root@jenkins ~]# ssh-copy-id -i .ssh/id_rsa.pub 10.0.0.7
```

jenkins项目中使用shell命令做打包推送动作

Dashboard > game_job > Configuration

Configure

- General
- 源码管理
- 构建触发器
- 构建环境
- Build Steps**
- 构建后操作

查看 可用的环境变量列表

```
DATE=$(date +%Y-%m-%d-%H-%M-%S)
CODE_DIR="./"
WEB_DIR="/code"

get_code_tar(){
    cd $CODE_DIR && tar zcf /opt/web-$DATE.tar.gz ./*
}

scp_code_web(){
    scp /opt/web-$DATE.tar.gz 10.0.0.7:$WEB_DIR
}

code_tarxf(){
    ssh 10.0.0.7 "cd $WEB_DIR && mkdir web-$DATE && tar xf web-$DATE.tar.gz -C web-$DATE"
}

ln_html(){
    ssh 10.0.0.7 "cd $WEB_DIR && rm -rf html && ln -s web-$DATE html"
}

main(){
    get_code_tar;
    scp_code_web;
    code_tarxf;
    ln_html;
}
```

```
1 Shell命令:
2 DATE=$(date +%Y-%m-%d-%H-%M-%S)
3 CODE_DIR="./"
4 WEB_DIR="/code"
5
6 get_code_tar(){
7     cd $CODE_DIR && tar zcf /opt/web-$DATE.tar.gz ./*
8 }
9
10 scp_code_web(){
11     scp /opt/web-$DATE.tar.gz 10.0.0.7:$WEB_DIR
12 }
13
14 code_tarxf(){
15     ssh 10.0.0.7 "cd $WEB_DIR && mkdir web-$DATE && tar xf web-$DATE.tar.gz -C web-$DATE"
16 }
17
18 ln_html(){
19     ssh 10.0.0.7 "cd $WEB_DIR && rm -rf html && ln -s web-$DATE html"
20 }
21
22 main(){
23
24     get_code_tar;
25     scp_code_web;
26     code_tarxf;
27     ln_html;
28 }
29 main
```

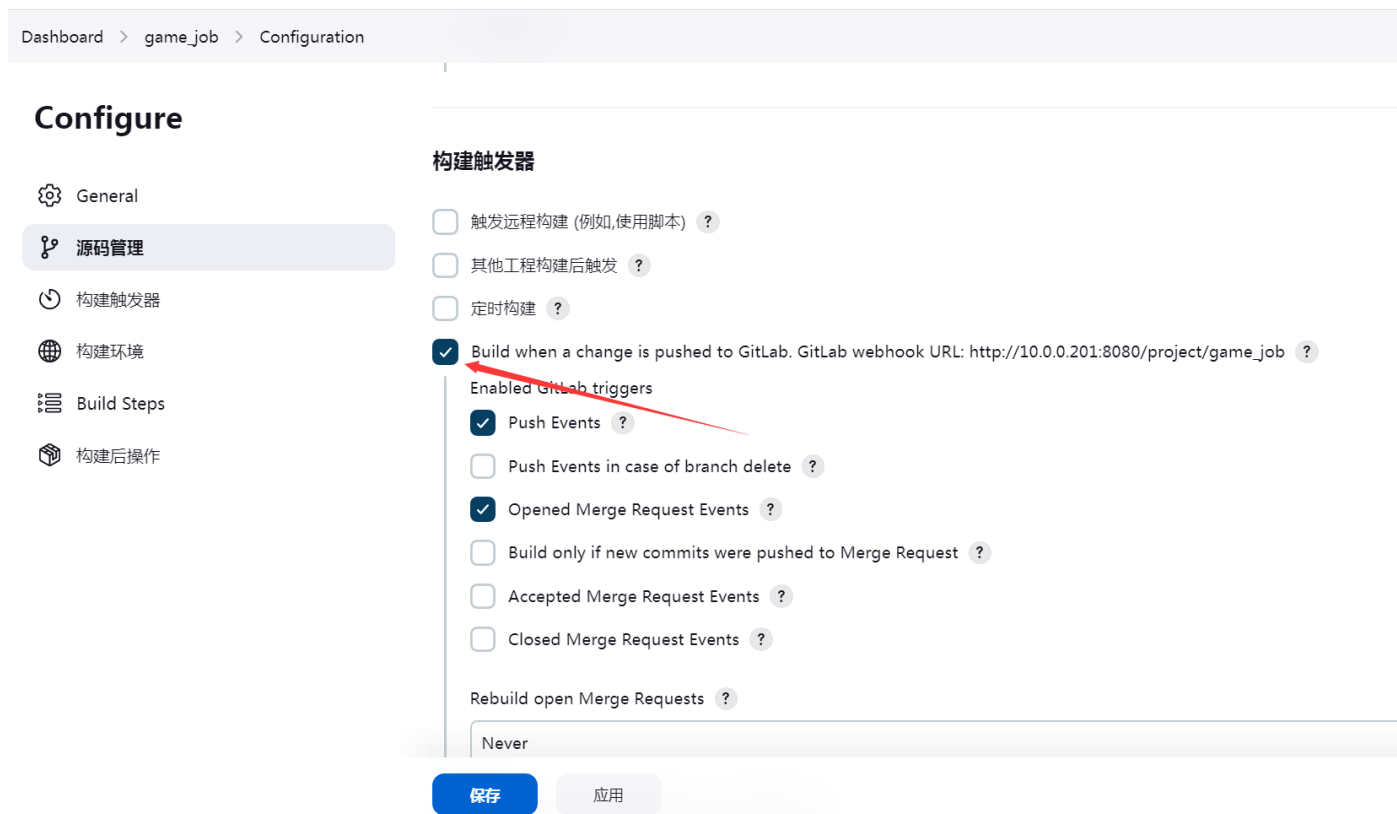
保存点击立即构建:

浏览器访问web服务器显示成功



03. 配置web-hook

1. 开启触发器



生成token并保存

Dashboard > game_job > Configuration

Configure

- General
- 源码管理**
- 构建触发器
- 构建环境
- Build Steps
- 构建后操作

☐ Cancel pending merge request builds on update ?

Allowed branches

- ☒ Allow all branches to trigger this job ?
- ☐ Filter branches by name ?
- ☐ Filter branches by regex ?
- ☐ Filter merge request by label

Secret token ?

da3c4bdcafd0f825f8265c533cd193cc

Generate

Clear

生成Secret Token

☐ GitHub hook trigger for GITScm polling ?

☐ 轮询 SCM ?

2.复制Token配置到Gitlab



ops游戏组 / Game

Auto DevOps流水线已启用。如果未找到CI配置文件，将使用该流水线。
此GitLab实例上尚未启用容器镜像库。请通知管理员启用以便Auto DevOps能够正常工作。

设置 更多信息

G Game

3 次提交 1 个分支 0 个标签 7.4 MB 项目存储

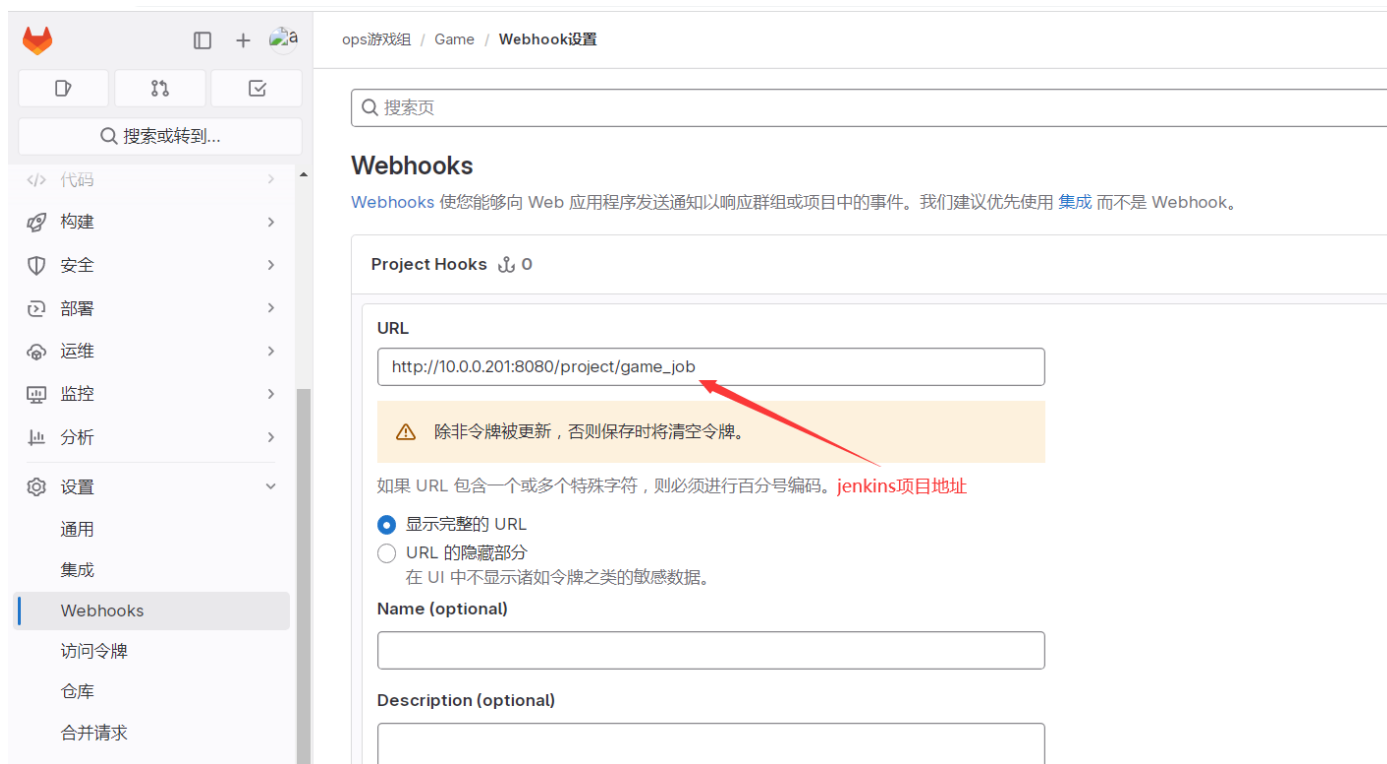
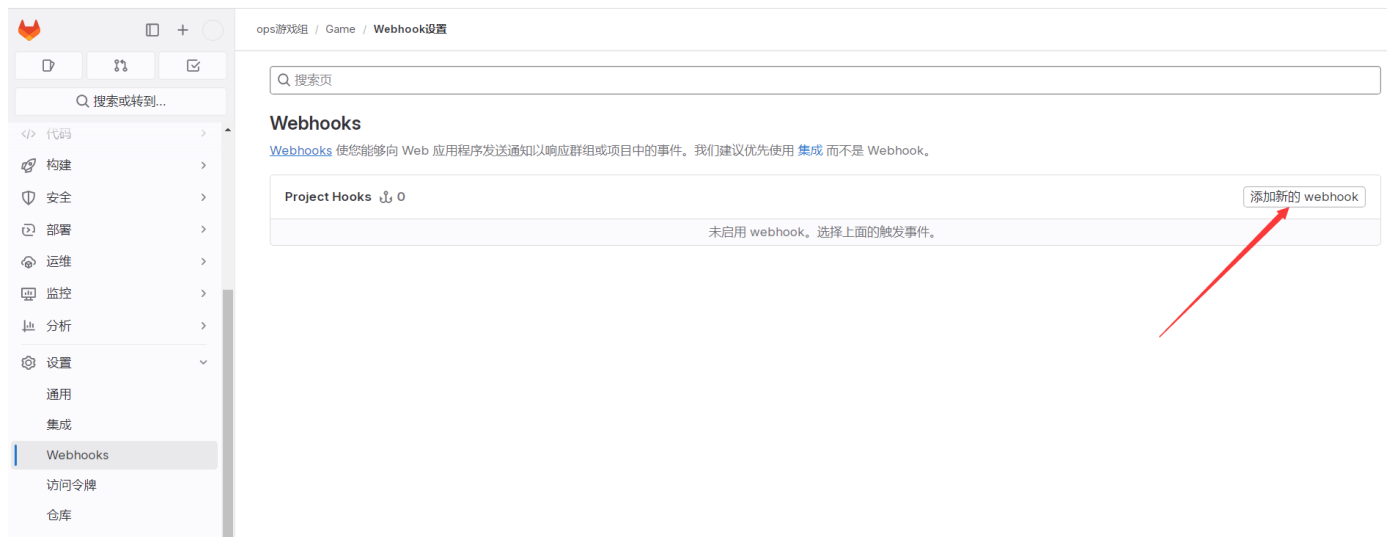
通用 集成 Webhooks 访问令牌 仓库 合并请求 CI/CD 软件包与镜像库 监控 使用量配额

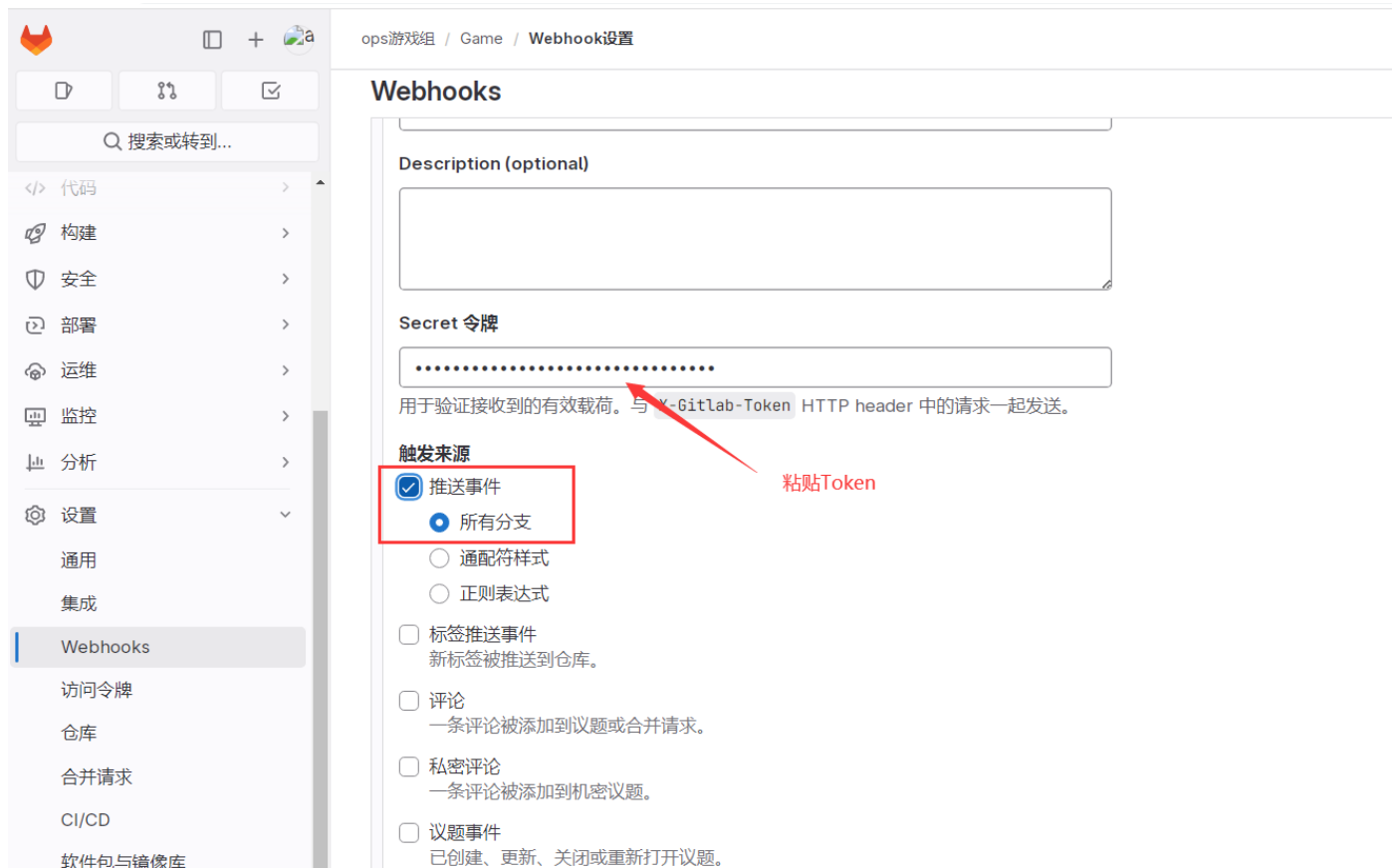
添加自述文件 添加LICENSE 添加更新日志 添加贡献信息

web钩子设置 最后提交

XX
XX
XX
XX
XX

roms





配置完成

测试: 修改代码推送到gitlab仓库的游戏项目中,会自动触发Jenkins拉取代码并将拉取到的修改后的代码推送到WEB服务器

04 .Jenkins集成SonarQube

一. jenkins上配置SonarQube服务端

系统管理->系统设置->sonarQube 告诉jenkins SonarQubeServer服务端地址

1.Name定义

2.URL添加SonarQube的地址

3token添加一个secret text 填写之前安装SonarQube的Token

SonarQube installations

List of SonarQube installations

Name

SonarQube

Server URL

Default is http://localhost:9000

http://10.0.0.203:9000/

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

- 无 -

+ 添加

点击保存，然后在进来点击添加配置token

保存

应用

类型

Secret text

范围 ?

全局 (Jenkins, nodes, items, all child items, etc)

Secret

.....

ID ?

sonar_jenkins

描述 ?

sonar

添加

取消

Dashboard > 系统管理 > System >

SonarQube installations

List of SonarQube installations

Name

SonarQube

Server URL

Default is http://localhost:9000

http://10.0.0.203:9000/

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonar

+ 添加

高级

选择sonar

保存

应用

二. Jenkins配置sonar客户端的位置

jenkins配置执行sonar-scanner命令的家目录，让jenkins能找到该命令 系统管理->全局工具配置

SonarQube Scanner 安装

新增 SonarQube Scanner

☰ SonarQube Scanner

Name

Sonar

SONAR_RUNNER_HOME

/usr/local/sonar

☐ 自动安装 ?

新增 SonarQube Scanner

保存

应用

三. 项目中集成SonarQube

Dashboard > game_job > Configuration

Configure

⚙ General

🔗 源码管理

🕒 构建触发器

🌐 构建环境

☰ Build Steps

📦 构建后操作

```
scp_code_web;  
code_tarxf;  
ln_html;  
}  
main
```

高级 ▾

增加构建步骤 ▴

Filter

Execute SonarQube Scanner
Invoke Ant
Invoke Gradle script
Run with timeout
Set build status to "pending" on GitHub commit
SonarScanner for MSBuild - Begin Analysis
SonarScanner for MSBuild - End Analysis
执行 Windows 批处理命令
执行 shell
调用顶层 Maven 目标

注意: 添加此选项后将扫描框拖拽到Shell步骤的上面

Dashboard > game_job > Configuration

Configure

General

源码管理

构建触发器

构建环境

Build Steps

构建后操作

Build Steps

Execute SonarQube Scanner

JDK ?
JDK to be used for this SonarQube analysis
(Inherit From Job)

Path to project properties ?

Analysis properties ?
sonar.projectName=\${JOB_NAME}
sonar.projectKey=html
sonar.sources=.

Additional arguments ?

保存

应用

05. Jenkins集成微信

一. 集成微信介绍

jenkins集成微信，测试环境流程，开发写完代码直接提交到gitlab，自动触发jenkins执行构建，运行结果可以直接展示在测试平台，每次发版的测试版本信息可以通过jenkins发送到企业微信通知相关项目人员。

二. 注册企业微信公众号

网址: <https://mp.weixin.qq.com/>

请选择注册的帐号类型



订阅号

具有信息发布与传播的能力
适合个人及媒体注册



服务号

具有用户管理与提供业务服务的能力
适合企业及组织注册



小程序

具有出色的体验，可以被便捷地获取与传播
适合有服务内容企业和组织注册



企业微信
原企业号

具有实现企业内部沟通与协同管理的能力
适合企业客户注册

选择完成后把下面信息进行完善->使用自己微信扫描二维码进行管理后-点注册

企业名称	jenkintest
填写企业、政府或组织名称	
行业类型	IT服务 > 计算机软件/硬件/信息服务 ▼
人员规模	1-50人 ▼

管理员信息

管理员姓名	李振亚
-------	----------------------------------

请填写企业微信管理员的姓名

管理员手机号	+ 86 ▼ 16687182731
--------	---

请输入你的手机号码

短信验证码	216525	获取验证码
-------	-------------------------------------	-----------------------

请输入手机短信收到的6位验证码

管理员微信	 扫码绑定微信，创建完成后请用此微信登录管理后台
-------	--

☒ 我同意并遵守 [《腾讯企业微信服务协议》](#) [《隐私政策》](#) [《红包使用授权协议》](#)

注册

注册成功登陆后点击应用管理->创建应用

上传一张log->填写应用名称-可见范围->创建应用-保存好agentId和secret写入py脚本

应用

小程序

硬件

应用

基础



打卡



审批



汇报



公告



文件盘



同事吧



公费电话 送



企业邮箱 送



微信红包封面



企业支付

自建



机器人



my-jenkinstest-job



创建应用



jenkins-message

暂无应用介绍

AgentId 1000003

Secret 6GLvHLQrjIFXAHwCfCIXMnLcmHCJBvZooGCZ2zKHZrk

可见范围 李振亚

管理员 李振亚

应用负责人 [设置](#) 将企业成员配置为应用负责人，成员即可在企业微信内管理此应用

功能

发送消息

使用管理工具中的“消息群发”或API发送消息

接收消息

接收用户发送的普通消息以及菜单操作、外部联系人变更回调等事件信息

自动回复

通过接收用户的消息，自动回复

三. 修改py脚本信息

```
1 #1. 上传脚本到/server/scripts
2 [root@jenkins ~]# mkdir -p /server/scripts
3 [root@jenkins ~]# cd /server/scripts
4 [root@jenkins scripts]#
5 [root@jenkins scripts]# ll
6 total 4
7 -rw-r--r-- 1 root root 2008 Oct 25 2019 jenkins_notify.py
8
```

```
[root@jenkins scripts]# egrep -n "touser|agentid|Secret|CorpId" jenkins_notify.py
12: def GetToken(CorpId, Secret):
15:     "corpId": CorpId,
16:     "corpsecret": Secret
34:     "touser": "LiZhenYa",
36:     "agentid": 1000002,
63:     CorpId = "wwa95c9738129a5c8e"
64:     Secret = "q7Q7IfUOKDws0WxnN0IF6vESqBpxpV6opVyuSKUxexY"
70:     Token = GetToken(CorpId, Secret)
```

发送给谁
收信人的id号
企业ID号
认证Secret

```

14 return array (
15     // 企业的id, 在管理端->"我的企业" 可以看到
16     "CORP_ID"          => "ww55ca070cb9b7eb22",
17
18     // "通讯录同步"应用的secret, 开启api接口同步后, 可以在管理端->"通讯录同步"看到
19     "CONTACT_SYNC_SECRET" => "ktmzrVI1UH0UW63zi7-JyzsgTL9NfwUhHde6or6zwQY",
20
21     // 某个自建应用的id及secret, 在管理端 -> 企业应用 -> 自建应用, 点进相应应用可以看到
22     "APP_ID"            => 1000002,
23     "APP_SECRET"        => "v1Z2KSw2WqPFECaWn2R0a1dFsanVF5sE4IE6X5ogveQ",
24
25     // 打卡应用的 id 及secrete, 在管理端 -> 企业应用 -> 基础应用 -> 打卡,
26     // 点进去, 有个"api"按钮, 点开后, 会看到
27     "CHECKIN_APP_ID"    => 3010011,
28     "CHECKIN_APP_SECRET" => "3Qz20GPvE1Eb6WKpEDfczvyQjL5Lr1CjrDTKn0RHdLE",
29
30     // 审批应用的 id 及secrete, 在管理端 -> 企业应用 -> 基础应用 -> 审批,
31     // 点进去, 有个"api"按钮, 点开后, 会看到
32     "APPROVAL_APP_ID"   => 3010040,
33     "APPROVAL_APP_SECRET" => "1vrlwItWpz_5Qkud55aImQPCvpzi51H3F2j-10QzhYE",
34 );

```

四. 本地执行脚本测试

```

1 yum -y install python-pip
2 wget https://bootstrap.pypa.io/pip/2.7/get-pip.py
3 python get-pip.py
4 pip2.7 install requests
5
6 [root@jenkins scripts]# python jenkins_notify.py 11 /etc/hosts game

```

五. 集成到Jenkins

```

1 复制执行命令
2 echo "=====Start Notify======"
3 echo ${SCM_CHANGELOG} > /tmp/${JOB_NAME}_change.log
4 python /server/scripts/jenkins_notify.py ${BUILD_URL} /tmp/${JOB_NAME}_change.log ${JOB_NAME}
5 rm -fv /tmp/${JOB_NAME}_change.log

```

构建后操作

Post build task

Tasks

Log text

Operation

-- AND --

新增

Script

```
echo "=====Start Notify====="  
echo ${SCM_CHANGELOG} >  
/tmp/${JOB_NAME}_change.log  
python /server/scripts/jenkins_notify.py  
${BUILD_URL} /tmp/${JOB_NAME}_change.log
```

Run script only if all previous steps were successful

☐

Escalate script execution status to job status

☐

Add another task

增加构建后操作步骤

06.Pipeline流水线

一. CI/CD持续集成/持续部署

1. 持续集成(Continuous integration)是一种软件开发实践，即团队开发成员经常集成它们的工作，通过每个成员每天至少集成一次，也就意味着每天可能会发生多次集成。每次集成都通过自动化的构建（包括编译，发布，自动化测试）来验证，从而尽早地发现集成错误。比如（你家装修厨房，其中一项是铺地砖，边角地砖要切割大小。如果一次全切割完再铺上去，发现尺寸有误的话浪费和返工时间就大了，不如切一块铺一块。这就是持续集成。）
2. 持续部署（continuous deployment）是通过自动化的构建、测试和部署循环来快速交付高质量的产品。某种程度上代表了一个开发团队工程化的程度，毕竟快速运转的互联网公司人力成本会高于机器，投资机器优化开发流程化相对也提高了人的效率。比如（装修厨房有很多部分，每个部分都有检测手段，如地砖铺完了要测试漏水与否，线路铺完了要通电测试电路通顺，水管装好了也要测试冷水热水。如果全部装完了再测，出现问题可能会互相影响，比如电路不行可能要把地砖给挖开.....。那么每完成一部分就测试，这是持续部署。）
3. 持续交付 Continuous Delivery:频繁地将软件的新版本，交付给质量团队或者用户，以供评审尽早发现生产环境中存在的问题；如果评审通过，代码就进入生产阶段比如（全部装修完了，你去验收，发现地砖颜色不合意，水池太小，灶台位置不对，返工吗？所以不如没完成一部分，你就去用一下试用验收，这就是持续交付。）
4. 敏捷思想中提出的这三个观点，还强调一件事：通过技术手段自动化这三个工作。加快交付速度。

二.pipeline介绍

1. 什么是pipeline

Jenkins 2.0的精髓是Pipeline as Code，是帮助Jenkins实现CI到CD转变的重要角色。什么是Pipeline，简单来说，就是一套运行于Jenkins上的工作流框架，将原本独立运行于单个或者多个节点的任务连接起来，实现单个任务难以完成的复杂发布流程。Pipeline的实现方式是一套Groovy DSL，任何发布流程都可以表述为一段Groovy脚本，并且Jenkins支持从代码库直接读取脚本，从而实现了Pipeline as Code的理念。

2. Pipeline 概念

Pipeline 是一个用户定义的 CD 流水线模式。Pipeline 代码定义了通常包含构建、测试和发布步骤的完整的构建过程。

3. Node

`node` 是一个机器，它是 `Jenkins` 环境的一部分，并且能够执行 `Pipeline`。同时，`node` 代码块也是脚本式 `Pipeline` 语法的关键特性。

4. Stage

`Stage` 块定义了在整个 `Pipeline` 中执行的概念上不同的任务子集（例如“构建”，“测试”和“部署”阶段），许多插件使用它来可视化或呈现 `Jenkins` 管道状态/进度。

5. Step

一项任务。从根本上讲，一个步骤告诉 `Jenkins` 在特定时间点（或过程中的“步骤”）要做什么。例如，使用 `sh step: sh 'make'` 可以执行 `make` 这个 `shell` 命令。